

Interpolable Nyquist criterion: fast stability charts and rightmost-root estimation for linear time-delay systems via adaptive phase integration

Daniel Bachrathy^a

^a*Department of Applied Mechanics, Faculty of Mechanical Engineering, Budapest University of Technology and Economics, Műegyetem rkp. 5., H-1111 Budapest, Hungary*

Abstract

A fast method is presented for computing stability charts of linear autonomous time-delay systems. A parameter point costs a few microseconds on one processor thread, and since the points are independent this cost divides almost exactly by the number of threads – from a single CPU core to tens of thousands of GPU cores – so that a full-HD chart takes about a millisecond on a graphics processor and an 8K chart can be recomputed in real time while a parameter slider is being moved. The number of unstable characteristic roots is obtained from the argument principle by tracking the phase of the characteristic function along the imaginary axis; the winding integral is reformulated as an ordinary differential equation and solved by an adaptive high-order integrator, with the integrand evaluated exactly by forward-mode automatic differentiation. An integration-free variant sums the exact phase increments between adaptively placed frequency samples (discrete phase unwrapping), with an ODE-style step-size control driven by the exact phase slope; it is one to two orders of magnitude faster, more robust near the stability boundary, and maps directly onto GPUs, in single and even half precision; its chart time follows a simple model – the single-thread cost per point times the number of points, divided by the effective parallelism of the hardware – measured on two CPUs and four GPUs. The adaptive frequency march makes practically unbounded integration ranges affordable, and the result is self-validating: the distance of the computed winding number from the nearest integer is a dimensionless error measure that scales with the requested tolerance, so a single tolerance setting transfers between systems without problem-specific tuning. During the same sweep, the characteristic root closest to the integration line – typically the rightmost root – is estimated as a by-product. This estimate turns the otherwise piecewise-constant root-counting map into an interpolable field, so a single chart simultaneously displays the number of unstable roots and the spectral gap of the stable domain; it drives the multi-dimensional bisection method to trace the true stability boundary at high resolution, locates the most robust parameter point via the largest inscribed circle, and can optionally be sharpened toward machine precision by higher-order corrections built on the same automatic derivatives. The characteristic function is extracted algorithmically from a simulation-ready right-hand side, including singular mass matrices (delay differential–algebraic systems). Retarded, neutral, distributed-delay, transcendental, high-dimensional and fractional-order case studies demonstrate the identical workflow, and comparisons with adaptive quadrature and semi-discretization quantify the efficiency. The method is available in the open-source Julia package `InterpolatedNyquist.jl`.

Keywords: time-delay systems, stability chart, Nyquist criterion, argument principle, neutral delay differential

1. Introduction

Time delays are inherent in a wide range of engineering systems: regenerative machine-tool vibrations [1, 2, 3, 4], digitally controlled and teleoperated machines, active vibration suppression, wheel shimmy, and traffic or population dynamics all lead to delay differential equations (DDEs) [5, 6, 7, 8]. Since the characteristic function $\mathcal{D}(\lambda)$ of even the simplest linear autonomous DDE is transcendental, it possesses infinitely many characteristic roots, and the stability of an equilibrium can rarely be decided in closed form. Engineering practice therefore relies on *stability charts*: maps over a plane of design parameters that show where the equilibrium is asymptotically stable [9, 10]. The practical value of such a chart depends on two properties of the underlying numerical method: how fast a single parameter point can be classified, and how much extra information (dominant root location, spectral gap, robustness margin) the classification yields.

A large family of methods operates in the time domain by discretizing either the solution operator or the infinitesimal generator of the delay system. The semi-discretization method of Insperger and Stépán [11, 12, 13, 9], the full-discretization method [14], Chebyshev collocation [15], pseudospectral differencing [16, 17], and the spectral element method [18] all construct a finite transition matrix whose eigenvalues approximate the characteristic multipliers or exponents; spectral methods to certify all roots in a prescribed right half-plane are also available [19]. Mature software packages implement these ideas: DDE-BIFTOOL [20, 21], TRACE-DDE [22], Knut [23, 24], and `SemiDiscretizationMethod.jl` [25], which also contains the recent multiplication-free variant with linear time complexity [26]. The great strength of this family is generality: time-periodic coefficients (e.g., milling) are handled by the same machinery through Floquet theory. For *autonomous* systems, however, this generality is paid for at every single parameter point: a large discretization matrix is built and its dominant eigenvalues are computed, although only the sign of the rightmost characteristic exponent – one bit of information – decides stability. Moreover, delayed systems of neutral type (where the essential spectrum destroys the convergence of many discretizations [27, 28]), systems with algebraic constraints, i.e., delay differential–algebraic equations (DDAEs) [29, 30], distributed delays [31], and fractional-order systems [32, 33, 34] each require dedicated, nontrivial extensions in the time domain.

The frequency domain offers a complementary route, since all the listed system classes share the same abstract object: an analytic characteristic function $\mathcal{D}(\lambda)$ whose roots in the right half-plane must be counted (for a comprehensive treatment see [35]). Classical tools include the D-subdivision method [36], the analytical stability criteria of Stépán [6], the Nyquist and Mikhailov criteria [37, 38], the direct method of Walton and Marshall [39], the Rekasius substitution [40], the cluster treatment of characteristic roots [41], and the frequency-sweeping tests of Li, Niculescu

Email address: bachrathy@mm.bme.hu (Daniel Bachrathy)

and Çela [42]; the Lambert- W formalism [43] and the matrix method of Louisell [44] target the dominant roots directly, the quasi-polynomial root finder QPmR [45, 46] computes all roots in a prescribed rectangle from the values of $\mathcal{D}(\lambda)$ on a grid – a contour-integral strategy also developed for general nonlinear eigenvalue problems by Beyn [47] – and the YALTA toolbox [48] automates root-locus and stability analysis for classical and fractional delay systems. A recent survey of argument-principle-based counting techniques is given in [49].

Two frequency-domain strategies are particularly relevant here. The first solves the two real equations $\text{Re } \mathcal{D}(i\omega) = \text{Im } \mathcal{D}(i\omega) = 0$ in the parameter space extended by the frequency ω , which yields the D-curves, i.e., the parameter combinations where a root crosses the imaginary axis. The multi-dimensional bisection method (MDBM) [50, 51] traces these one-dimensional solution manifolds very efficiently. The D-curves, however, are *not* the stability boundaries: they merely mark root crossings, and the number of unstable roots on either side remains unknown – and automating that decision is surprisingly hard (triangulation into closed cells, filtering of spurious slivers). The turning stability-lobe diagram illustrates a more fundamental inefficiency: towards low spindle speeds the lobe structure becomes infinitely dense, and almost all D-curves lie deep inside the unstable domain (cf. Appendix Appendix A) [52, 53, 54], so the method spends most of its effort on curves that never bound the stable region.

The second strategy counts the unstable roots directly by the argument principle, i.e., by the winding number of $\mathcal{D}(i\omega)$ – the generalized Nyquist, or Mikhailov, criterion [6, 55]. Root counting by contour integrals of $\mathcal{D}'/\mathcal{D}$ has a long history for general analytic functions [56, 57, 58]; for delay systems, Hassard [59] derived a semi-analytical counting formula, Zítek and Vyhřídál formulated argument-increment criteria for neutral systems [60, 61], and, closest to the present work, Xu and Wang [62] and Xu, Stépán and Wang [63] evaluated the argument-principle “testing integral” numerically, observing that a rough quadrature suffices because the result must be an integer. The appeal of root counting is that the answer $Z(\mathbf{p})$ is exactly the quantity a stability chart needs. Its curse is twofold. First, $Z(\mathbf{p})$ is a piecewise-constant integer field: it jumps abruptly across the stability boundaries, so it cannot be interpolated, and brute-force charts need fine parameter grids. Second, the numerical evaluation is hardest *exactly at the stability boundary* – the only region one actually cares about – because as a root approaches the integration line the integrand of the winding integral develops a near-singular peak whose width shrinks to zero, and fixed grids or rough quadratures either miss the peak or need excessive refinement; the infinite frequency range poses an additional truncation problem.

This paper revisits the integration-based Nyquist criterion and turns it into a fast and robust tool with *interpolable* output. Its speed is best stated in a hardware-independent way: a parameter point costs a few microseconds on one processor thread, and this cost divides almost exactly by the number of threads, from one CPU core to tens of thousands of GPU cores, so that a full-HD chart is computed in about a millisecond and an 8K chart in real time on a current graphics processor. The contributions are the following. (i) The winding integral is reformulated as a scalar ordinary differential equation for the phase, marched by a high-order adaptive integrator [64, 65, 66] whose step-size control resolves the near-singular peaks and crosses the practically unbounded frequency range, with the integrand evaluated exactly by forward-mode automatic differentiation [67]. (ii) Since the count needs only the end value of the continuous phase, the integration can be dropped altogether: the exact phase increments between adaptively placed

65 samples are summed (discrete phase unwrapping), and the exact phase slope from the same automatic derivatives
 drives an ODE-style step-size control that rejects any step hiding a root transition. This integration-free march needs
 one evaluation of $\mathcal{D}$ per step and tens of evaluations per parameter point, is one to two orders of magnitude faster
 than the phase ODE on the CPU and more robust near the boundary, and – consisting of identical scalar steps on
 a fixed-size state – maps directly onto GPUs, in single and even half precision. Measured on two CPUs and four
 70 GPUs, its chart time scales linearly with the number of threads up to the saturation of the device, and a two-parameter
 model (single-thread cost and effective parallelism) predicts it for any hardware. (iii) The location of the characteristic
 root closest to the integration line – normally the rightmost root – is estimated *during the same sweep* at negligible
 extra cost; the first-order estimate is the smooth, sign-correct default, and an optional local polish (Taylor, Newton
 or a combined mode) sharpens it toward machine precision for a modest fraction of the cost when an accurate root
 75 is wanted. Tracking several local minima yields multiple roots at once. This ordered march is what distinguishes the
 phase ODE from plain quadrature, whose recursive subdivision visits the frequencies out of order and cannot recognize
 the minima at all. (iv) The root estimate makes the chart interpolable: a combined coloring shows the integer number
 of unstable roots outside and the smooth spectral gap inside the stable domain, which is a strikingly effective visual
 diagnostic. (v) The integrality of the computed winding number provides a built-in, dimensionless error estimate
 80 that scales with the requested solver tolerance and lets the accuracy of the count be prescribed in advance; where it
 is structurally blind – a skipped peak costs exactly $\pm\pi$ and leaves the result an integer – the independently obtained
 root supplies the missing check. (vi) The estimated root distance serves as a scalar objective for MDBM, which then
 traces the *true* stability boundary (not the D-curves) at very high equivalent resolution, and the most robust parameter
 point is found as the center of the largest circle inscribed in the stable domain. (vii) The characteristic function is
 85 extracted algorithmically from a simulation-ready right-hand side – including singular mass matrices (DDAEs) – so
 the user never has to derive $\mathcal{D}(\lambda)$ by hand. (viii) The method applies verbatim to retarded, neutral, distributed-delay,
 transcendental, high-dimensional and (preliminarily) fractional-order systems, and is released as the open-source
 Julia [68] package `InterpolatedNyquist.jl`. Compared with the rough-quadrature testing integral of [62, 63],
 the present formulation adds adaptive error control tied to the integer-residual estimate, the rightmost-root by-product
 90 that enables interpolation and boundary tracing, and a far wider class of demonstrated applications; compared with
 contour-moment methods such as ZEAL [58], it needs no bounded search region and is tailored to the half-line
 stability sweep.

The paper is organized as follows. Section 2 summarizes the system classes and the argument-principle count-
 ing formula. Section 3 presents the phase ODE, the rightmost-root tracking, the higher-order refinement, the self-
 95 validating error estimate, and the integration-free discrete phase unwrapping. Section 4 assembles these ingredients
 into fast stability charts. Section 5 describes the automatic extraction of $\mathcal{D}(\lambda)$ from a time-domain model. Section 6
 walks through the complete workflow on a constrained two-degree-of-freedom system with delayed control, Section 7
 quantifies accuracy, CPU time and memory, including a comparison with the semi-discretization method, Section 8
 measures how the cost divides among CPU and GPU threads and what that means for real-time charts and for number

100 formats, and Section 9 concludes. Appendix Appendix A collects twelve further case studies from all supported system classes, and Appendix Appendix B lists the discrete march in algorithmic form.

2. Linear autonomous delay systems and the argument principle

2.1. System classes and the characteristic function

Consider the linear autonomous delay system in first-order descriptor form

$$\mathbf{E} \dot{\mathbf{x}}(t) = \mathbf{A}_0 \mathbf{x}(t) + \sum_{j=1}^m (\mathbf{A}_j \mathbf{x}(t - \tau_j) + \mathbf{B}_j \dot{\mathbf{x}}(t - \tau_j)) + \int_0^{\theta_{\max}} \mathbf{W}(\vartheta) \mathbf{x}(t - \vartheta) d\vartheta, \quad (1)$$

105 with state $\mathbf{x}(t) \in \mathbb{R}^N$, constant matrices $\mathbf{E}, \mathbf{A}_j, \mathbf{B}_j \in \mathbb{R}^{N \times N}$, discrete delays $\tau_j > 0$ and a distributed-delay kernel $\mathbf{W}$. The exponential ansatz $\mathbf{x}(t) = \xi e^{\lambda t}$ leads to the characteristic matrix and the characteristic function

$$\mathbf{M}(\lambda) = \lambda \mathbf{E} - \mathbf{A}_0 - \sum_{j=1}^m (\mathbf{A}_j + \lambda \mathbf{B}_j) e^{-\lambda \tau_j} - \int_0^{\theta_{\max}} \mathbf{W}(\vartheta) e^{-\lambda \vartheta} d\vartheta, \quad \mathcal{D}(\lambda) = \det \mathbf{M}(\lambda). \quad (2)$$

Equation (1) covers all the system classes treated in this paper: *retarded* DDEs ($\mathbf{E} = \mathbf{I}, \mathbf{B}_j = \mathbf{0}$), *neutral* DDEs ($\mathbf{B}_j \neq \mathbf{0}$), *delay differential–algebraic* equations (DDAEs, singular $\mathbf{E}$, e.g., due to kinematic constraints or algebraic force balances), and *distributed* delays. Beyond (1), the method below requires only an analytic function $\mathcal{D}(\lambda)$ that
110 can be evaluated at complex arguments; hence transcendental characteristic functions of continua (e.g., cosh-type expressions of elastic beams [69]) and fractional-order quasi-polynomials containing terms λ^α , $\alpha \notin \mathbb{N}$, are admissible as well.

The equilibrium of (1) is asymptotically stable if all roots of $\mathcal{D}(\lambda) = 0$ satisfy $\operatorname{Re} \lambda < 0$. More generally, a guaranteed decay rate (spectral gap) $|\sigma|$ – often called γ -stability – requires all roots to lie left of the shifted line
115 $\operatorname{Re} \lambda = \sigma < 0$. We denote by $Z(\mathbf{p})$ the number of roots with $\operatorname{Re} \lambda > \sigma$ for the design parameters $\mathbf{p}$; the stability chart is the map $\mathbf{p} \mapsto Z(\mathbf{p})$, and the stable (or γ -stable) domain is the region $Z = 0$.

For large $|\lambda|$ the characteristic function grows like $\mathcal{D}(\lambda) \sim c \lambda^n$, where the leading order n equals N for retarded systems in state-space form but is smaller for DDAEs (the algebraic rows remove dynamic content) and is only an *effective, possibly non-integer* order for neutral ($\mathcal{D} \sim \lambda^n (1 + \sum_j b_j e^{-\lambda \tau_j})$) and fractional systems. The counting formula
120 below needs n , and Section 3.1 shows that it can be estimated numerically without any structural knowledge.

Fractional orders and the branch cut. Fractional powers are the one class in which the choice of integration line is a question of validity rather than of cost. The principal branch of λ^μ is analytic on the complex plane cut along the negative real axis, and $\mathcal{D}$ inherits that cut (measured across it, $\mathcal{D}$ jumps by 14.9 for the fractional controller of Appendix Appendix A, while across the imaginary axis the jump is identically zero). Since the cut lies entirely
125 outside the closed right half-plane, the choice $\sigma = 0$ is admissible – which is what makes the whole class available to the method. The branch *point* at $\lambda = 0$ lies on the contour, but the resulting integrand singularity $\sim \omega^{\mu-1}$ is integrable for $\mu > 0$ and contributes nothing provided $\mathcal{D}(0) \neq 0$; when $\mathcal{D}(0) = 0$ the origin is itself a root on the contour and

the package reports the invalid marker of Section 3.4 rather than inventing a number. A *shifted* line ($\sigma < 0$), however, encloses the segment $[\sigma, 0]$ of the cut, the argument principle does not apply, and Appendix Appendix A demonstrates what that costs.

2.2. Counting unstable roots on a half-line

Let the integration line $\lambda = \sigma + i\omega$ be free of characteristic roots, and let $\mathcal{D}$ be analytic with finitely many roots to the right of the line. Applying the argument principle to the D-shaped contour composed of the line and the arc at infinity, and exploiting the conjugate symmetry $\mathcal{D}(\bar{\lambda}) = \overline{\mathcal{D}(\lambda)}$ of real-coefficient systems, the number of roots to the right of the line is obtained from frequencies $\omega \geq 0$ only [6, 59]:

$$Z = \frac{n}{2} - \frac{1}{\pi} \Delta\theta, \quad \Delta\theta = \int_0^\infty \theta'(\omega) d\omega, \quad \theta(\omega) = \arg \mathcal{D}(\sigma + i\omega), \quad (3)$$

where θ is continuous in ω and $n/2$ accounts for the phase gathered on the arc at infinity, where $\mathcal{D} \sim c\lambda^n$. This is the half-line (Mikhailov-type) form of the Nyquist criterion. Writing $\mathcal{D}'(\omega) \equiv \partial_\omega \mathcal{D}(\sigma + i\omega)$ for the derivative along the line, the phase slope is

$$\theta'(\omega) = \operatorname{Im} \frac{\mathcal{D}'(\omega)}{\mathcal{D}(\sigma + i\omega)} = \frac{\operatorname{Re} \mathcal{D} \cdot \partial_\omega \operatorname{Im} \mathcal{D} - \operatorname{Im} \mathcal{D} \cdot \partial_\omega \operatorname{Re} \mathcal{D}}{|\mathcal{D}|^2}. \quad (4)$$

For $\sigma = 0$, Z counts the unstable roots; for $\sigma < 0$ it counts the roots violating the prescribed spectral gap, so the same formula draws γ -stability charts.

Three standing assumptions deserve explicit statement. (i) *Finitely many roots right of the line, none on it.* For retarded systems and regular DDAEs of low index this holds for every σ greater than some abscissa; for DDAEs of higher index the spectrum can acquire advanced-type root chains that run into the right half-plane, and the count is then infinite for any line [70, 30] – such systems are outside the scope of (3), although the diagnostic of Section 3.4 flags them, because the truncated integral then fails to settle to an integer. (ii) *Neutral tail.* For neutral systems $\mathcal{D} \sim c\lambda^n(1 + \sum_j b_j e^{-\lambda\tau_j})$ and the phase of the parenthesis oscillates without decay as $\omega \rightarrow \infty$: the integral in (3) does not converge but oscillates within a bounded envelope. If the neutral amplitude satisfies $\beta = \sum_j |b_j| < 1$ – the strong-stability condition of the difference operator [27, 28] – that envelope is bounded by $\arcsin(\beta)/\pi < 1/2$ per delay term, so truncating at a generic $\omega_{\max}$ and rounding still recovers the exact count. As $\beta \rightarrow 1$ (essential instability approaching) the bound saturates and the integer residual of Section 3.4 grows accordingly – the diagnostic degrades exactly where the answer itself becomes fragile, which is the honest behaviour (see the acceleration-feedback case study A.6 in Appendix Appendix A). (iii) *Analyticity right of the line.* Fractional powers λ^α are taken on the principal branch, whose cut lies on the negative real axis; for $\sigma \geq 0$ the region right of the line is then free of branch points, and (3) applies verbatim. A shifted line with $\sigma < 0$, however, encloses part of the cut, so the argument principle does not apply there; the fractional case studies of Appendix Appendix A show how the violation announces itself.

2.3. Why the naive evaluation is impractical

Formula (3) looks like a textbook quadrature problem, but three properties make its direct use for stability charts inefficient. First, $Z(\mathbf{p})$ is integer-valued: it is constant inside every cell of the chart and jumps at the boundaries, so the

map cannot be interpolated, and any grid-based chart must be fine enough to resolve every cell of interest. Second,
 160 the numerical evaluation is hardest exactly where it matters most. If a root $\lambda_r = \sigma_r + i\omega_r$ lies close to the integration
 line, the integrand (4) develops a near-Lorentzian peak at $\omega \approx \omega_r$,

$$\theta'(\omega) \approx \frac{-(\sigma_r - \sigma)}{(\sigma_r - \sigma)^2 + (\omega - \omega_r)^2} + \text{smooth part}, \quad (5)$$

whose half-width $|\sigma_r - \sigma|$ tends to zero as the parameter point approaches the stability boundary, while its integral
 contribution stays $\pm\pi$; any method that samples the integrand pointwise and happens to place all its samples outside
 such a peak miscounts Z by exactly one, and – since the result stays an integer – does so without any warning. Third,
 165 the integration range is infinite: the phase slope decays only algebraically, with delay-induced oscillatory ripples, so
 a fixed truncation $\omega_{\max}$ must balance truncation error against the cost of resolving a ripple over a huge interval.

Section 3 addresses the first and third difficulties with the step-size control of an adaptive integrator, and turns the
 second into an asset by extracting the location of the offending root from the very peak that makes the integration hard.
 The peak-skipping hazard, by contrast, cannot be legislated away by any choice of integrator; what can be done – and
 170 what Section 3.4 does – is to detect it, using the fact that the root estimate and the phase count come from independent
 mechanisms and must agree.

3. Ordered adaptive integration with rightmost-root tracking

3.1. The winding integral as an ordinary differential equation

The key step is disarmingly simple: instead of applying quadrature to (3), the accumulated phase is treated as the
 175 state of the scalar initial value problem

$$y'(\omega) = \theta'(\omega) = \text{Im} \frac{\mathcal{D}'(\omega)}{\mathcal{D}(\sigma + i\omega)}, \quad y(0) = 0, \quad \tilde{Z} = \frac{n}{2} - \frac{y(\omega_{\max})}{\pi}, \quad Z = \text{round}(\tilde{Z}), \quad (6)$$

which is handed to a modern adaptive integrator with local error control. It should be stated plainly what this formula-
 tion is and is not. The right-hand side of (6) does not depend on the state y , so the Jacobian $\partial_y y'$ vanishes identically:
 (6) is a quadrature problem in disguise, it is never stiff in the classical sense, and an automatically stiffness-switching
 integrator provably never switches on it. What the integrator contributes is therefore not stiffness handling but *adap-*
 180 *tive step-size control on an ordered march*: steps contract around the near-singular peaks (5), where the phase turns
 by $\pm\pi$ over a frequency interval of width $|\sigma_r - \sigma|$, and expand again over the smooth stretches. Because the integrand
 is analytic, a high-order explicit Runge–Kutta pair is the appropriate choice; the default is Vern9 [64], which at the
 default tolerance is three orders of magnitude more accurate than the fifth-order alternative [65] for a factor of 2.4 in
 cost (Section 7.1). The ordered march is what makes the root tracking of Section 3.2 possible, which is the reason the
 185 phase ODE is preferred here over plain quadrature (Section 3.5).

The integrand is evaluated *exactly*: the characteristic function is called once per frequency with a dual-number
 argument $\lambda = \sigma + i(\omega + \delta)$, $\delta^2 = 0$, so forward-mode automatic differentiation [67] delivers $\mathcal{D}$ and $\mathcal{D}' = \partial_\omega \mathcal{D}$

simultaneously, without finite differencing and without any user-supplied derivative. A tiny offset $\omega \mapsto \max(\omega, 10^{-9})$ regularizes branch points of fractional terms at the origin.

Two practical parameters remain. The truncation frequency $\omega_{\max}$ can be chosen generously: past the highest structural resonance the phase slope decays algebraically, the adaptive step size grows accordingly, and a *too small* $\omega_{\max}$ is immediately visible in the result, because the truncated tail shifts $\tilde{Z}$ away from an integer (Section 3.4). The cost of the tail is governed by the decay of the delay-induced phase ripple. If the delayed terms sit at least two orders below the leading power of $\mathcal{D}$ – as for delayed *position* feedback on mechanical systems – the ripple decays at least like ω^{-2} and pushing $\omega_{\max}$ far beyond the resonances is essentially free: for the fourth-order benchmark the march costs 1426, 1474 and 1506 evaluations of $\mathcal{D}$ at $\omega_{\max} = 10^6, 10^8$ and 10^{10} respectively – four extra decades for six percent more work. If a delayed term sits only one order below the leading power (delayed *velocity* feedback), the ripple decays merely like ω^{-1} , the solver keeps resolving it (Fig. 2c), and $\omega_{\max}$ does cost. The resulting recommendation is asymmetric: *for accuracy, be generous* ($\omega_{\max} = 10^6$ is a safe default for every system in this paper); *for speed, be frugal* – two to three decades above the highest resonance leaves a truncated tail of order $0.2/\omega_{\max}$ in $\tilde{Z}$, far below the rounding threshold, and for a slowly decaying ripple cuts the evaluation count by more than an order of magnitude. Since a too-small $\omega_{\max}$ announces itself in the residual, the choice can be made empirically rather than by analysis.

The leading order n in (6) is estimated numerically, and the natural place to do so is not the imaginary axis at all. The term $n/2$ accounts for the phase gathered on the arc at infinity, and that arc lies in the *right half-plane*, where every delayed factor $e^{-\lambda\tau_j}$ decays exponentially. Probing along the positive real axis therefore measures precisely the quantity the counting formula needs, with the delay terms already extinguished:

$$n \approx \underset{s_k \in [s_*/10, s_*]}{\text{LS-slope}} \left(\ln s_k \mapsto \ln |\mathcal{D}(s_k)| \right), \quad s_* \gg 1, \quad (7)$$

the least-squares slope of the log–log magnitude over a handful of log-spaced real samples. The probe is decoupled from the integration range (n is a property of $\mathcal{D}$ alone; the default is $s_* = 10^8$ whatever $\omega_{\max}$ is), costs a handful of evaluations once per sweep, and converges as $O(1/s_*)$ – measured here as $7.5 \cdot 10^{-5}, 7.4 \cdot 10^{-7}$ and $7.4 \cdot 10^{-9}$ at $s_* = 10^4, 10^6$ and 10^8 – including the effective integer order of neutral equations and the non-integer order of fractional ones. The real axis is essential here: on the imaginary axis $|e^{-i\omega\tau}| = 1$, so the delay terms superimpose a ripple on $\ln |\mathcal{D}(i\omega)|$ that a fit aliases rather than averages, and for a neutral system, whose ripple never decays, a single-point estimator diverges in proportion to ω (measured: $n_{\text{est}} = 80\,028$ at $\omega = 10^6$ for a system whose true effective order is 2). Probing where the delays are exponentially small removes all of this at a stroke, and in parameter sweeps the estimate is computed once when n is parameter-independent.

Two limitations are worth naming. First, $|\mathcal{D}(s)| \sim s^n$ overflows for large n , so s_* is reduced automatically until $|\mathcal{D}|$ is finite, which for very high-order determinants caps the achievable accuracy. The better answer is structural rather than numerical: whenever the model offers a natural open-loop/feedback split, the *return difference* $\mathcal{D} = \det(\mathbf{I} + \mathbf{Q}_0^{-1}\mathbf{F}) = 1 + L(\lambda)$ has the same right-half-plane zeros, adds poles only at the (stable) open-loop roots, and tends to 1 at infinity – then $n = 0$ *exactly* and the enormous magnitudes disappear; case study A.9 in Appendix Appendix

A quantifies the difference. Second, Equation (7) presumes that $\mathcal{D}$ is polynomially bounded in the right half-plane, which holds for every quasi-polynomial but not for the transcendental characteristic functions of continua, where $\cosh(\sqrt{\lambda^2 + c\lambda}L)$ grows like $e^{\lambda L}$. The package detects this automatically (for a genuine power law the fitted slope is reproduced one window lower; for exponential growth it is not) and falls back to a fit along the imaginary axis, where such functions are bounded ($n \approx 0$ for the beam of Appendix Appendix A).

The requested relative and absolute solver tolerances control the local error of the accumulated phase, measured in radians. Since the exact winding number is an integer multiple of π in the phase, the error of $\tilde{Z}$ inherits the tolerance directly as a *dimensionless* quantity: the same default (10^{-5}) is appropriate for a machining model and for a dense 50×50 matrix determinant alike, with no problem-dependent scaling (Section 3.4).

3.2. Rightmost-root estimation as a by-product

The peaks of the integrand are not only a numerical nuisance – they carry the information where the dominant characteristic roots are. Along the integration line, $|\mathcal{D}(\sigma + i\omega)|^2$ dips to a sharp local minimum at $\omega \approx \omega_r$ for every nearby root $\lambda_r = \sigma_r + i\omega_r$, and the smallest minimum belongs to the root closest to the line, which for $\sigma = 0$ is typically the rightmost root that decides stability. The solver therefore tracks $\min_{\omega} |\mathcal{D}|^2$ during the march at negligible cost: at every integrand evaluation the already computed quantities $\mathcal{D}$ and $\mathcal{D}'$ yield the real part of one Newton step towards the root,

$$\hat{\sigma} = \sigma + \operatorname{Re}\left(-\frac{\mathcal{D}}{d\mathcal{D}/d\lambda}\right) = \sigma - \frac{\operatorname{Re} \mathcal{D} \cdot \partial_{\omega} \operatorname{Im} \mathcal{D} - \operatorname{Im} \mathcal{D} \cdot \partial_{\omega} \operatorname{Re} \mathcal{D}}{|\mathcal{D}'|^2}, \quad (8)$$

using $d\mathcal{D}/d\lambda = -i\mathcal{D}'$, and the frequency of the minimum is stored as $\hat{\omega}$. The pair $(\hat{\sigma}, \hat{\omega})$ is a first-order estimate of the dominant root location returned together with Z ; its accuracy is highest exactly at the stability boundary, where the root lies on the line and the Newton step becomes exact up to the solver tolerance. Away from the boundary the estimate degrades gracefully, but there only its sign and magnitude matter – as a smooth measure of the distance to instability.

Several roots can be tracked simultaneously. A compile-time variant of the solver detects every local minimum of $|\mathcal{D}|^2$ through the sign change of its ω -derivative, $2(\operatorname{Re} \mathcal{D} \partial_{\omega} \operatorname{Re} \mathcal{D} + \operatorname{Im} \mathcal{D} \partial_{\omega} \operatorname{Im} \mathcal{D})$, and maintains the N_r best candidates in a sorted, allocation-free buffer; a boundary test at $\omega = 0$ captures real dominant roots, since $|\mathcal{D}|^2$ is even in ω . Tracking one root is the default and adds a few percent to the sweep time; tracking zero roots gives maximal sweep speed. One caveat deserves emphasis: the *deepest* minimum of $|\mathcal{D}|$ belongs to the rightmost root near the stability boundary, but deeper inside the stable or unstable domains a different root branch may produce a deeper dip. When the true dominant root is required away from the boundary, several minima should be tracked ($N_r \approx 5\text{--}10$), all of them refined, and the dominant root taken as the one with the maximal real part – this recipe is validated against an independent eigenvalue method in Section 7.4.

That recommendation is worth more than a footnote, because the single-minimum estimate is not merely inaccurate away from the boundary: it is *discontinuous*. Where one branch overtakes another as the closest one, $\hat{\omega}$ jumps between modes and $\hat{\sigma}$ jumps with it, and a method that fits a local linear model – as the boundary tracer of Section 4.2

does inside each cell – will read the jump as a zero crossing and place a spurious boundary point in the middle of the stable domain. The dominant root passes through a branch crossing smoothly, because it is a maximum over branches rather than the nearest one, so building the coloring and the boundary objective from it removes the artefact at its source. For the count itself the caveat is immaterial, since near the boundary the crossing root always produces the global minimum.

3.3. Optional higher-order root refinement

The raw tracked estimate $(\hat{\sigma}, \hat{\omega})$ – one Newton step off the minimum of $|\mathcal{D}|$ – is the method’s *default* root output, and deliberately so. It is first-order accurate, but it is the *smoothest* field over a parameter chart (a single formula evaluated at every pixel), it is the cheapest, and its *sign* – all the stability classification needs – is already correct. When an accurate rightmost root is wanted, the same automatic derivatives drive an optional local polish. Writing the Taylor expansion at $\hat{\lambda} = \hat{\sigma} + i\hat{\omega}$,

$$\mathcal{D}(\hat{\lambda} + \Delta) \approx \mathcal{D} + \mathcal{D}_\lambda \Delta + \frac{1}{2} \mathcal{D}_{\lambda\lambda} \Delta^2 + \frac{1}{6} \mathcal{D}_{\lambda\lambda\lambda} \Delta^3 = 0, \quad (9)$$

where the subscripts denote derivatives with respect to λ evaluated at $\hat{\lambda}$, the package offers (i) *Newton–Raphson* iterations $\Delta = -\mathcal{D}/\mathcal{D}_\lambda$; (ii) a single *quadratic* (Poly-2) or (iii) *cubic* (Poly-3) Taylor solve, where the root of the local polynomial closest to $\hat{\lambda}$ is selected via the eigenvalues of its companion matrix; and (iv) a *combined* mode that runs the polynomial and Newton polishes and keeps, per root, the candidate with the smallest $|\mathcal{D}|$. The first derivative is exact (automatic differentiation); the higher derivatives are central differences of that exact first derivative, which avoids the compilation cost of nested dual numbers. Each polish is *monotone*: it is accepted only if it lowers $|\mathcal{D}|$ below the seed, otherwise the raw estimate is returned, so refinement can help but never degrade the estimate.

Which polish is best is genuinely system-dependent: on a smooth quasi-polynomial the Newton iteration reaches machine precision, but on a rational characteristic function, near the poles of the plant frequency response, a Newton step can settle on a neighbouring root and speckle an otherwise smooth chart, where the Taylor polishes stay put. The combined mode is the robust choice – lowest residual on every system tested – at about 25% over the raw solve; the trade-offs are measured in Section 6.4. The practical guidance matches the design goal of a single, generally valid setting: leave refinement off for charts, turn it on when a specific rightmost root matters, and read a precise decay-rate *map* from the σ -level contours of Section 4.4 rather than from a per-pixel polish.

If several estimates are tracked, the same refinement applied to each returns a set of dominant roots, comparable to the output of dedicated root finders such as QPmR [45] – with the difference that here the root solve is a cheap, optional post-processing step of the stability count.

3.4. Self-validating error estimate

The exact winding number is an integer, so

$$\varepsilon = |\tilde{Z} - \text{round}(\tilde{Z})| \quad (10)$$

is a rigorous, freely available indicator of the accumulated numerical error: it contains the local integration errors, the
 285 tail truncated at $\omega_{\max}$, and the residual of the estimated order (7). (When a characteristic root lies on the integration line
 itself, no count is defined; the package then returns an invalid-count marker rather than a number.) Since the phase
 is integrated in radians, ε is *dimensionless* and scales essentially linearly with the requested tolerance – amplified
 by the accepted-step count, since the per-step error tolerance accumulates along the march – hence the tolerance
 acts as a universal accuracy dial that transfers between systems without re-tuning. In the numerical experiments of
 290 Section 6.3, ε tracks the tolerance across six decades: at 10^{-2} the residual reaches order one and genuine miscounts
 appear – the deliberate failure demonstration of Fig. 4 – while 10^{-4} already keeps ε an order of magnitude below the
 rounding threshold $1/2$ and produces reliable charts at near-maximal speed, and 10^{-8} yields reference-quality values
 at a moderate CPU-time premium; the package default 10^{-5} adds a further safety decade at little cost.

Supplying the leading order. The order n is an *input* of the counting formula (3), not a result of it, and the estimator
 295 exists only so that a user who does not know n is not blocked. Whenever n is known by inspection of the model – and
 it usually is – it should simply be passed (`n_power_max`): it is then exact rather than estimated, and it costs nothing.
 Non-integer fractional orders are legitimate inputs too.

When the residual stops being a check. The diagnostic has one precondition: the *convergence of the tail*. $\tilde{Z}$ is only
expected to be an integer when the truncated remainder $\int_{\omega_{\max}}^{\infty}$ is negligible. For retarded systems it is, and ε behaves
 300 as described: across the retarded case studies of Appendix Appendix A the median residual is $\sim 10^{-4}$ and essentially
 no grid point is uncertain. For *neutral* systems the integrand does not decay at all: the tail contributes an oscillation
 bounded by $\arcsin(\beta)/\pi$ rather than a vanishing remainder (Section 2.2), and the residual inherits it – on the neutral
 panels of Appendix Appendix A the median residual is ≈ 0.1 , with the counts nevertheless correct. For that class ε is
not a usable self-check; what protects the count is the analytic bound $\arcsin(\beta)/\pi < 1/2$ instead, and enlarging $\omega_{\max}$
 305 does not help, because the oscillation is bounded, not decaying.

One caveat is inherited from the argument principle itself and bounds what ε can certify. A march that steps *over* a
 peak loses exactly $\pm\pi$ of phase, so $\tilde{Z}$ shifts by exactly one and remains just as close to an integer as before: ε measures
 the *quadrature* quality and is structurally blind to a skipped peak. Nor is this hypothetical: approaching a stability
 boundary the peak half-width $|\sigma_r - \sigma|$ collapses to many orders of magnitude below any practical step size (Table 5),
 310 and the marching solver then strides past it and reports success. No pointwise method is immune (Section 7.1).

What rescues the situation is the second output of the same sweep. The tracked root of Section 3.2 is obtained
 from the minima of $|\mathcal{D}|^2$, a mechanism entirely independent of the accumulated phase, and it remains accurate exactly
 where the peak is narrow – that is, exactly where the count is fragile. The two therefore cross-validate each other: a
 skipped peak shows up as an *inconsistency* between $\text{sign}(\hat{\sigma} - \sigma)$ and the predicate $Z = 0$; both quantities are standard
 315 outputs of every sweep, so the check costs one comparison.

The test must, however, be restricted to a neighbourhood of the line: far from the boundary the tracked minimum
 need not belong to the *dominant* root (Section 3.2), so the tracked root may disagree with $Z > 0$ entirely legitimately

– applied blindly, the test fires on 459 of the 8000 points of a *fully converged* chart of the showcase system. A peak can only be stepped over where a root lies close to the line, so the informative test is

$$\text{sign}(\hat{\sigma} - \sigma) \text{ disagrees with } [Z = 0] \quad \text{and} \quad |\hat{\sigma} - \sigma| < h, \quad (11)$$

320 with h a small fraction of the spread of $\hat{\sigma}$ over the chart. With $h = 0.001$ – well below the median $|\hat{\sigma}| \approx 0.041$ of those spurious disagreements – the test fires on only 2 of the same 8000 points. Its scope should be stated precisely. Skipping one peak only ever changes Z by one, so the stable/unstable classification a chart displays is affected only where the *last* unstable root crosses – precisely the regime in which $\hat{\sigma}$ is small and the test is sharp; this is the silent error mode the residual cannot see, and at practical tolerances it is the only one left (Section 6.3). What the test does
325 not promise is to rescue a chart run far outside the rule of thumb: at $\text{tol} = 10^{-2}$ the accumulated phase error itself reaches order one, miscounts appear everywhere – not only near the boundary – and the near-boundary test catches only 10 of the 89 classification flips of such a run. What announces *that* failure is the residual field itself, which lights up over the whole chart (Section 6.3); the correct reaction is to tighten the tolerance, not to trust any pointwise flag. The same mechanism that flags a skipped peak can also mend it: the minimum of $|\mathcal{D}|^2$ *locates* the peak precisely
330 however narrow it is, and Section 6.3 shows how an optional callback uses that location to repair the count within the march itself.

3.5. Alternative integrators within the same framework

The phase integrand (4) can also be fed to classical adaptive quadrature; the package provides a Gauss–Kronrod back-end (QuadGK.jl [71]) with the identical interface. For the root count alone it is fully competitive – within a
335 factor of about three of the march in evaluations in either direction (Table 12) – and a reasonable choice whenever only Z is wanted. The decisive difference concerns the rightmost root. Adaptive quadrature subdivides its interval recursively and visits the frequencies in an essentially arbitrary order: there is no sweep along which a local minimum of $|\mathcal{D}|^2$ could be recognized. It is this ordered traversal – not accuracy, not speed, and certainly not stiffness – that earns the phase ODE its place as the default, because every chart in this paper is built from the tracked root: the
340 interpolable coloring (15), the boundary objective (16), the refinement seeds, and the consistency check of Section 3.4 all consume $\hat{\sigma}$, not Z alone. A third back-end evaluates the integral by a fixed-step trapezoidal rule with essentially zero overhead. It is included as a real-time floor, but a *uniform* frequency grid cannot resolve the peak (5) of a root close to the line, so its mean count error converges with the step count while its *worst* error over a chart does not (Section 7.5). Adaptivity is thus not a convenience here but the very mechanism that makes the count trustworthy.
345 The fourth back-end, introduced next, keeps the adaptivity and the ordered traversal but drops the integration itself.

3.6. Discrete phase unwrapping: counting without integration

The phase ODE (6) integrates θ' only to obtain one number, the end value $y(\omega_{\max}) = \Phi(\omega_{\max}) - \Phi(0)$ of the continuous phase $\Phi(\omega) = \arg \mathcal{D}(\sigma + i\omega)$. That number is available *without* integration. Between two samples $\omega_a < \omega_b$

the phase increment is known exactly up to a multiple of 2π ,

$$\Delta_{ab} = \text{Arg} \frac{\mathcal{D}(\sigma + i\omega_b)}{\mathcal{D}(\sigma + i\omega_a)} \in (-\pi, \pi], \quad (12)$$

and it equals the true increment $\Phi(\omega_b) - \Phi(\omega_a)$ whenever the latter lies in $(-\pi, \pi)$. On any sampling $0 = \omega_0 < \omega_1 < \dots < \omega_K = \omega_{\max}$ that is dense enough in this sense, the counting formula (3) becomes the finite sum

$$\tilde{Z} = \frac{n}{2} - \frac{1}{\pi} \sum_{k=1}^K \Delta_{k-1,k}, \quad (13)$$

which is *exact*: there is no integration error, only the requirement that no branch of the phase is lost. This is the classical phase-unwrapping problem, with one decisive advantage over its signal-processing namesake: the same dual-number evaluation that returns $\mathcal{D}$ also returns the exact phase slope $\theta' = \text{Im}(\mathcal{D}'/\mathcal{D})$ at every sample.

Adaptivity.. The slope makes the march adaptive in exactly the way an embedded Runge–Kutta pair is. A trial step $h = \omega_b - \omega_a$ evaluates $\mathcal{D}$ and $\mathcal{D}'$ at ω_b and compares the exact (observed) increment with the trapezoidal value predicted by the exact slopes at both ends,

$$e = \left| \Delta_{ab} - \frac{h}{2}(\theta'_a + \theta'_b) \right|, \quad (14)$$

accepts the step if $e \leq \text{tol}$ (in radians; default 0.3), and sets the next trial step to $h \cdot \min\{4, \max\{0.2, 0.9(\text{tol}/e)^{1/3}\}\}$, the exponent reflecting the $O(h^3\theta''')$ trapezoid error – the standard step-size controller, with the exact increment in the role of the higher-order solution. On smooth stretches e follows the curvature of the phase. Across an unresolved root transition – a peak (5) of width $|\sigma_r - \sigma|$ hidden inside the step – the observed increment contains a jump of almost $\pm\pi$ that the end-point slopes do not predict, $e \approx \pi$, and the step is rejected and refined until the transition is resolved. In the tail, where $\Phi \approx n\pi/2$ plus a small ripple, $e \rightarrow 0$ and the step grows geometrically, so the frequency axis is crossed in a logarithmic number of steps whatever $\omega_{\max}$ is. The delay-induced ripple, which keeps an integrator busy for thousands of steps when a delayed term sits only one order below the leading power (Section 3.1), becomes invisible as soon as its phase amplitude drops below the tolerance, because its contribution is contained *exactly* in the end-point phase. Each step costs a single evaluation of $\mathcal{D}$ (a Vern9 step costs sixteen), and on the systems of this paper a point needs 30–130 evaluations instead of 10^3 – 10^4 (Section 7.3). Automatic differentiation is still essential: the error estimate (14) needs the exact slope, and the root tracking below needs $\mathcal{D}'$.

Root tracking without extra evaluations.. The rightmost-root by-product of Section 3.2 carries over. A minimum of $|\mathcal{D}|$ inside an accepted step is recognized by the sign change of $\partial_\omega|\mathcal{D}|$ between its end points; it is located on the cubic Hermite interpolant of the complex function $\mathcal{D}(\omega)$ built from $\mathcal{D}$ and $\mathcal{D}'$ at both ends – $\mathcal{D}$ itself is smooth even where its phase is not – and one Newton step (8) from the located minimum gives $(\hat{\sigma}, \hat{\omega})$, under the same trust region as the refinement of Section 3.3. No additional evaluation of $\mathcal{D}$ is spent.

375 *Below the resolution of the number format..* When a root lies so close to the line that its transition is narrower than the smallest representable step ($\approx 8 \epsilon_{\text{mach}} \omega$), the step cannot be refined further and the observed increment is ambiguous near $\pm\pi$. The branch is then decided by the *side* of the root: a root right of the line lowers the phase by π , one left of it raises it, and the side is the sign of the Newton estimate from the current sample – the peak repair of Section 6.3 at no cost. Such a count is a decision rather than a measurement, and the point is flagged. Since the sum (13) carries no
 380 integration error, the residual ε of (10) now contains only the truncated tail and the error of the order n : it is typically 10^{-7} – 10^{-10} , and a value above $1/4$ flags a root *on* the line (e.g. $\mathcal{D}(\sigma) = 0$, $\tilde{Z} = k + \frac{1}{2}$).

Limitations.. An end-point test cannot see an increment of exactly 2π : two transitions in the same direction inside one step escape it. Two situations produce them in practice. First, a *rational* $\mathcal{D}$ places a pole next to every lightly damped mode; when a root leaves that mode into the right half-plane, the pole–zero pair winds the phase by -2π
 385 within a frequency interval of the size of the modal damping. Writing $\mathcal{D}$ as an entire function – multiplying out the (stable) denominators, which does not change the count in $\text{Re } \lambda > \sigma$ – removes the pair. Second, a regenerative delay τ can place a chain of roots near the axis, spaced $\approx 2\pi/\tau$ apart in frequency; capping the step at $h \leq \pi/(2\tau_{\text{max}})$ over the resonance band removes the ambiguity at a moderate cost. With these two rules the discrete march reproduces the reference counts on every point of the three benchmark charts, and near a Hopf boundary it is more robust than the
 390 phase ODE (Table 10).

Fitness for parallel hardware.. The march consists of identical scalar steps on a small, fixed-size state, one evaluation each, with no allocation and no solver infrastructure. It therefore maps directly onto a GPU kernel with one thread per parameter point, and even onto narrow number formats for the evaluation of $\mathcal{D}$, since only the *phase* of $\mathcal{D}$ matters and the tolerance of (14) is a fraction of a radian (Section 8). The algorithm and its parameters are listed in
 395 Appendix Appendix B.

4. Fast stability charts

4.1. Brute-force sweep with interpolable coloring

The single-point solve of Section 3 is embarrassingly parallel over the parameter plane; a multithreaded sweep over a grid $\mathbf{p}_{ij}$ returns the fields Z , $\tilde{Z}$, $\min |\mathcal{D}|$, $\hat{\sigma}$ and $\hat{\omega}$. The integer field Z alone is a poor plotting quantity – flat
 400 plateaus separated by jumps – but the estimated root location repairs precisely this deficiency. The combined field

$$C(\mathbf{p}) = \begin{cases} \hat{\sigma}(\mathbf{p}), & \text{if } Z(\mathbf{p}) = 0, \\ Z(\mathbf{p}), & \text{otherwise,} \end{cases} \quad (15)$$

is a remarkably efficient coloring trick for visual inspection: inside the stable domain ($Z = 0$, $\hat{\sigma} < 0$) it varies *smoothly* and displays the spectral gap, i.e., how far the dominant root lies from the imaginary axis, while outside it shows the integer number of unstable roots as discrete color levels. One glance at a single map thus reveals where the system is

stable, how robustly stable it is there, and how many roots must be stabilized elsewhere. Because C is smooth over
 405 the stable domain, interpolation between grid points is meaningful exactly where it is useful, so a coarse and cheap
 grid suffices for the colored background; only the integer levels of the unstable side are plotted as flat colors.

4.2. High-resolution boundary tracing with MDBM

The stability boundary itself deserves better than pixel resolution. The multi-dimensional bisection method
 (MDBM) [50, 51] locates the zero set of a scalar function over a coarse initial mesh and refines only the bracket-
 410 ing cells, halving the mesh size in every iteration. Here *only the estimated root distance is used* as the scalar objective:

$$g(\mathbf{p}) = \text{sign}([Z(\mathbf{p}) = 0] - \tfrac{1}{2}) |\hat{\sigma}(\mathbf{p}) - \sigma|, \quad (16)$$

which is positive inside the stable (γ -stable) domain, negative outside, crosses zero exactly on the stability boundary,
 and – unlike Z itself – varies almost linearly across it, because near the boundary $\hat{\sigma}$ is an accurate estimate of the
 dominant root’s real part (Section 3.2). MDBM can therefore interpolate the zero crossing within each bracketing cell
 to sub-cell accuracy. One detail is essential rather than incidental: $\hat{\sigma}$ here must be the *dominant* root of Section 3.2,
 415 not the single closest minimum – the latter is discontinuous at branch crossings, and a linear model fitted inside a cell
 straddling the jump interpolates a phantom boundary point in the interior of the stable domain, corrupting anything
 built on the point cloud, such as the inscribed circle of Section 4.3. Note the contrast with applying MDBM to the pair
 $\text{Re } \mathcal{D} = \text{Im } \mathcal{D} = 0$ in the parameter–frequency space: that approach traces *all* root-crossing D-curves, most of which
 lie deep in the unstable region (Section 1), while the objective (16) follows only the *true* stability limit. By the very
 420 nature of the bisection refinement, the computational effort concentrates exclusively on the boundary: k refinement
 levels raise the equivalent resolution of an $n_0 \times n_0$ initial mesh to $2^k n_0 \times 2^k n_0$, while the number of function evaluations
 grows only in proportion to the boundary length. On the fourth-order benchmark of Section 7, four refinement levels
 from a 20×20 mesh trace the boundary at a 320×320 -equivalent resolution from under 2000 evaluations (Table 8) –
 some 50 times fewer than the 10^5 a brute-force grid of that density would need.

4.3. The most robust parameter point

With a high-resolution boundary and a smooth interior metric available, the natural design question – *which*
parameter point is the most robustly stable? – reduces to a geometric one: the center of the largest circle inscribed
 in the stable domain maximizes the distance to the stability boundary in parameter space. The package computes it
 from the MDBM boundary point cloud (or from the binary stable mask of the brute-force grid) by a coarse-to-fine
 430 search followed by an omnidirectional pattern refinement with adaptive step control. Anisotropic parameter scaling
 turns the circle into an ellipse, which avoids degenerate results in elongated stable regions and lets the user weight the
 expected perturbations of the individual parameters. The exact radius is rarely of engineering interest; the *location* of
 the optimum is, and that location is insensitive to the residual discretization of the boundary. If a spectral-gap-optimal
 (rather than parameter-robust) point is preferred, the interpolable interior field $\hat{\sigma}(\mathbf{p})$ of (15) can be minimized instead
 435 – both options are cheap post-processing steps on data already computed.

4.4. σ -contours: decay-rate maps

Repeating the boundary tracing of Section 4.2 for a family of shifted lines $\sigma \in \{0, \sigma_1, \sigma_2, \dots\}$ produces the level curves of the guaranteed decay rate, i.e., a γ -stability map: each contour bounds the parameter region whose rightmost root lies left of $\text{Re } \lambda = \sigma$. The interior coloring by $\hat{\sigma}$ and the traced σ -contours are two views of the same robustness landscape – the former is a fast estimate on a coarse grid, the latter a high-resolution guaranteed boundary – and plotting them together provides a strong visual cross-validation of both (Section 6).

4.5. Recommended workflow

The complete chart-generation recipe used throughout the paper is:

1. *Colored sweep.* Multithreaded evaluation on a grid – with the discrete march of Section 3.6 for an entire $\mathcal{D}$, with the phase ODE (6) otherwise; plot the combined field (15). Cost: tens of milliseconds for a 100×100 grid with the discrete march (on a GPU, milliseconds for a full-HD grid), up to a few seconds with the phase ODE.
2. *Boundary tracing.* MDBM on the scalar objective (16), typically 4 refinement levels from a 20×20 – 30×30 initial mesh. Cost: comparable to or below the sweep.
3. *Robust point.* Largest inscribed circle/ellipse from the MDBM point cloud. Cost: milliseconds.
4. *Optional.* σ -contour family; root refinement at selected points; higher tolerance for publication-quality boundaries.

Every step reuses the same single-point solver, so the entire chart refreshes fast enough for interactive parameter exploration (Section 7).

5. Automatic extraction of the characteristic function from a simulation model

In engineering practice, a system studied for stability almost always exists as a *simulation model* first: a right-hand side function written for a general-purpose integrator such as `DifferentialEquations.jl` [66]. Deriving the characteristic function (2) from it by hand is tedious and error-prone, particularly for many degrees of freedom, several delays, or algebraic constraints. The package therefore extracts $\mathcal{D}(\lambda)$ *algorithmically* from the very same right-hand side $\mathbf{f}(\mathbf{u}, \mathbf{h}, \mathbf{p}, t)$ used for time-domain simulation, where $\mathbf{h}(\mathbf{p}, t - \tau)$ is the standard history-function interface for the delayed states.

The mechanism requires a single evaluation of $\mathbf{f}$ per requested λ . The state is seeded with dual-number perturbations, $\mathbf{u} = \mathbf{u}_{\text{eq}} + \mathbf{a}$, where the components of $\mathbf{a}$ carry independent infinitesimal directions, and the history argument is replaced by the *mock exponential history*

$$\mathbf{h}(\mathbf{p}, t) = \mathbf{u}_{\text{eq}} + \mathbf{a} e^{\lambda t}. \quad (17)$$

Whatever delayed value $\mathbf{h}(\mathbf{p}, t - \tau_j)$ the model requests contributes the factor $e^{-\lambda \tau_j}$ automatically, so discrete, multiple, and (approximated) distributed delays are all captured without declaring them. Evaluating $\mathbf{f}$ at $t = 0$ and collecting

the dual partial derivatives of the residual

$$r = \lambda \mathbf{E} \mathbf{a} - f(\mathbf{u}_{\text{eq}} + \mathbf{a}, \mathbf{h}, \mathbf{p}, 0) \quad \Rightarrow \quad \mathbf{M}(\lambda) = \frac{\partial \mathbf{r}}{\partial \mathbf{a}}, \quad \mathcal{D}(\lambda) = \det \mathbf{M}(\lambda), \quad (18)$$

yields the exact characteristic matrix of the *linearized* dynamics around $\mathbf{u}_{\text{eq}}$: the dual seeding performs the linearization, so the right-hand side may be fully *nonlinear* – no separate linearization step is required (the equilibrium itself could also be located from the same right-hand side, which is outside the scope of this paper; a residual check warns if the supplied $\mathbf{u}_{\text{eq}}$ is not an equilibrium). The mass matrix $\mathbf{E}$ is supplied separately – it is not part of the right-hand side, exactly as in the ODEFunction/DDEFunction convention of `DifferentialEquations.jl` – and it may be *singular*: rows of zeros in $\mathbf{E}$ turn the corresponding rows of $\mathbf{M}(\lambda)$ into the algebraic constraint equations, so delay differential–algebraic systems are handled by the identical code path. The determinant is evaluated with statically sized arrays, so a full extraction costs one right-hand-side call plus an $N \times N$ determinant, and the dual seeding composes transparently with the outer dual numbers of the phase ODE (6).

The complete user workflow is thus reduced to providing the simulation right-hand side, the mass matrix if it differs from the identity, and the parameter ranges of the chart:

```
function rhs(u, h, p, t)                # simulation model (DDE interface)
    P, D = p
    x1_del = h(p, t - 0.5; idxs=1)      # delayed position
    v1_del = h(p, t - 0.5; idxs=4)      # delayed velocity
    ...                                  # equations of motion + constraints
end

D(lam, p) = get_D_from_model(rhs, lam, p, Val(7); mass_matrix = E)
                                     # Val(7): the model has 7 states;
                                     # E is the (singular) mass matrix

Z, Zraw, minD, sigma_est, omega_est =
    calculate_unstable_roots_direct(D, (2.0, 1.0))
```

The leading order n required by the counting formula (3) is estimated automatically by (7) – including the effective orders of neutral characteristic functions and the non-integer orders of fractional ones – so no additional structural input is needed. In Section 6 the extraction is verified against a hand-derived characteristic function: the two agree to machine precision.

5.1. What the convenience costs

Automatic extraction is a convenience, and it is not free. Every evaluation of the extracted $\mathcal{D}$ runs the user’s right-hand side once with dual-number arguments and then takes a 7×7 determinant of the descriptor matrix, where the

Table 1: The price of never deriving $\mathcal{D}$ by hand. Both forms are the *same* function – they agree to machine precision and every one of the 10^4 counts is identical – and differ only in how each evaluation is obtained. Same system, same chart, same settings ($\omega_{\max} = 10^4$, default tolerance).

form of $\mathcal{D}$	chart (100×100)	per point [μ s]
hand-derived (no determinant, no AD)	1.97 s	197
automatic extraction from the RHS	41.47 s	4147
<i>ratio</i>	21×	21×

hand-derived form of the same system evaluates a flat scalar expression – a handful of multiplications, one exponential, no determinant and no automatic differentiation. Table 1 measures the difference on the identical chart: 1.97 s against 41.47 s for 100×100 points, a factor of about 21.

The trade is explicit, and both sides of it are defensible. If the model exists only as a simulation right-hand side – the normal situation early in a design study – the extracted form gives a correct chart in a minute with no derivation whatsoever. If the same chart is to be refreshed interactively or recomputed inside an optimization loop, the algebra is worth doing once: the hand-derived form turns the same chart into a sub-second one. The recommended workflow is exactly that order – explore with the extracted form, derive $\mathcal{D}$ by hand once the model has settled and speed starts to matter. All figures in this paper use the hand-derived form, having first verified it against the extracted one (Section 6).

6. Showcase: a constrained two-degree-of-freedom system with delayed PD control

All elements of the method are now demonstrated on a single mechanical example that is simple enough to verify by hand yet exercises every nontrivial feature at once: multiple degrees of freedom, a feedback delay, and an algebraic constraint, i.e., a delay differential–algebraic system with a singular mass matrix.

6.1. Model and automatic extraction

The main structure of mass m_1 is statically unstable (inverted-pendulum-like, modeled by the negative net suspension stiffness $k_1 < 0$ and damping c_1); it carries an absorber unit of mass m_2 attached through a spring and damper (k_2, c_2), and an instrumentation block of mass m_3 is *rigidly bolted* to the absorber mass. Instead of eliminating the rigid connection by hand, the model keeps it as an algebraic constraint with the internal force F_c – exactly the form in which a simulation model of a larger assembly would be written. A digital controller measures the motion of the main mass and applies the delayed PD force $u(t) = -P x_1(t-\tau) - D \dot{x}_1(t-\tau)$ to it. With the state vector $\mathbf{z} = (x_1, x_2, x_3, v_1, v_2, v_3, F_c)$,

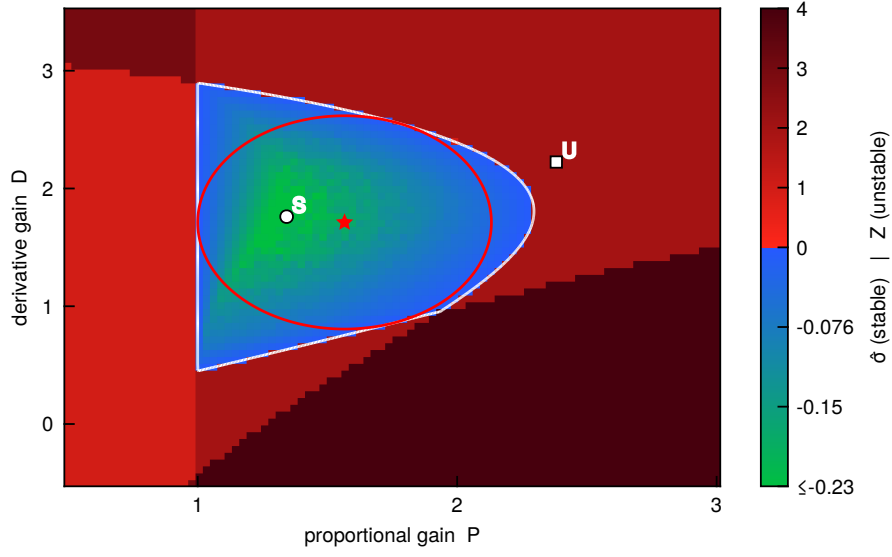


Figure 1: Stability chart of the constrained two-degree-of-freedom system with delayed PD control. Colored background: combined field (15) of the brute-force sweep (smooth spectral gap $\hat{\sigma}$ inside the stable domain, integer number of unstable roots outside). Black curve: stability boundary traced by MDBM with the scalar objective (16). Red ellipse and star: largest inscribed ellipse and the most robust parameter point. The markers S and U denote the stable and unstable points analyzed in Fig. 2.

the descriptor form (1) reads

$$\underbrace{\text{diag}(1, 1, 1, m_1, m_2, m_3, 0)}_{\mathbf{E}} \dot{\mathbf{z}}(t) = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ -k_1 x_1 - c_1 v_1 + k_2(x_2 - x_1) + c_2(v_2 - v_1) + u(t) \\ -k_2(x_2 - x_1) - c_2(v_2 - v_1) + F_c \\ -F_c \\ x_3 - x_2 \end{pmatrix}, \quad (19)$$

with the numerical values $m_1 = 1$, $m_2 = 0.3$, $m_3 = 0.2$, $k_1 = -1$, $k_2 = 1$, $c_1 = c_2 = 0.05$ and $\tau = 0.5$. The last row of (19) carries no time derivative – the mass matrix is singular – and the model is passed to the extraction of Section 5 exactly as written, in its simulation (right-hand-side) form. For verification, the constraint is also eliminated by hand ($x_3 = x_2$, merged mass $m_{23} = m_2 + m_3$), which yields the reduced quasi-polynomial

$$\mathcal{D}_{\text{red}}(\lambda) = \det \begin{pmatrix} m_1 \lambda^2 + (c_1 + c_2) \lambda + k_1 + k_2 + (P + D \lambda) e^{-\lambda \tau} & -(c_2 \lambda + k_2) \\ -(c_2 \lambda + k_2) & m_{23} \lambda^2 + c_2 \lambda + k_2 \end{pmatrix}. \quad (20)$$

The ratio of the automatically extracted 7×7 determinant to $\mathcal{D}_{\text{red}}$ is constant in λ to machine precision (deviation below 10^{-15} in the computed charts), which validates the mass-matrix extraction path end to end; the estimated leading order (7) is $n \approx 4.000$, as expected for two mechanical degrees of freedom. Having verified the two forms

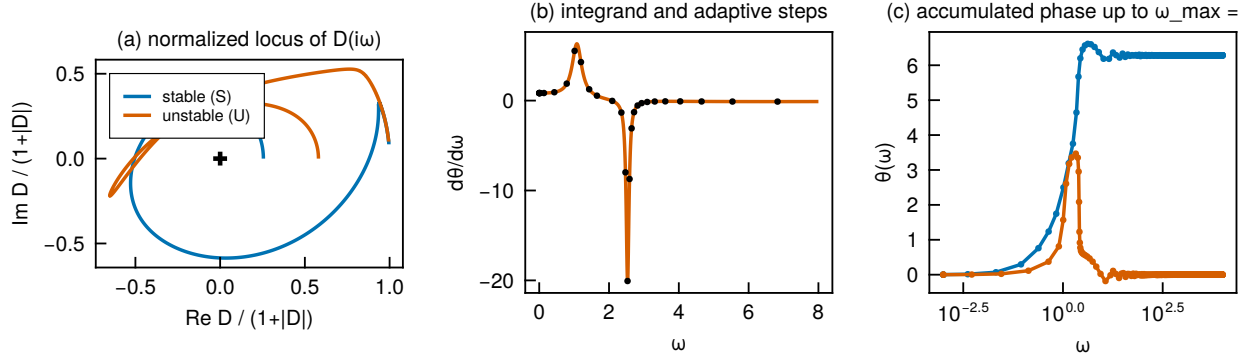


Figure 2: The method at the stable point S and unstable point U of Fig. 1. (a) Normalized locus of $\mathcal{D}(i\omega)$: the stable curve avoids the origin, the unstable one encircles it. (b) Phase slope $\theta'(\omega)$ with the accepted steps of the adaptive solver (dots): steps cluster automatically at the near-singular peaks that indicate close characteristic roots. (c) Accumulated phase up to $\omega_{\max} = 10^4$ on a logarithmic frequency axis: past the resonances the solver keeps resolving the slowly decaying (ω^{-1}) delay ripple of this velocity-feedback system with moderate, growing steps.

identical, the results below use the cheaper hand-derived form (20); what the automatic extraction costs instead is measured in Section 5.1.

6.2. The chart and the phase integration at work

Figure 1 shows the resulting stability chart over the control-gain plane (P, D) . Since the uncontrolled structure is unstable, the stable domain is a bounded island: its left boundary is the static (divergence) limit, while the remaining boundaries are Hopf-type curves whose crossing frequencies are delivered by $\hat{\omega}$. The combined coloring (15) displays the interior spectral gap – deep blue marks the fastest-decaying configurations – and the integer plateaus $Z = 1, 2, 3, 4$ outside, all in one map. The black MDBM curve is the high-resolution stability boundary traced from the scalar objective (16), and the red star marks the most robust gain combination found by the largest inscribed ellipse.

Figure 2 dissects the single-point computation at the two marked points. At the stable point S the locus of $\mathcal{D}(i\omega)$ avoids the origin and the accumulated phase settles at $\Delta\theta = n\pi/2$, giving $\tilde{Z} \approx 0$; at the unstable point U the locus encircles the origin and $\tilde{Z} \approx 2$. Panel (b) makes the central numerical argument of this paper visible: the accepted steps of the adaptive solver concentrate at the sharp peaks of θ' near the lightly damped resonances – precisely the features a fixed frequency grid tends to miss. Panel (c) shows the same march continuing over more than two further decades of frequency: the delayed *velocity* feedback of this system produces a phase ripple decaying only like ω^{-1} (Section 3.1), so the solver keeps taking moderate steps through the tail. The chart studies below use the standard window $\omega_{\max} = 10^4$ – its truncated tail, $\approx 0.2/\omega_{\max} \approx 2 \cdot 10^{-5}$, is far below the rounding threshold, and its role in the error budget is measured explicitly – while the single-point convergence study of Section 7.1 removes even that floor by integrating to $\omega_{\max} = 10^6$. Either way the truncation is exposed by the integer-residual diagnostic, never hidden.

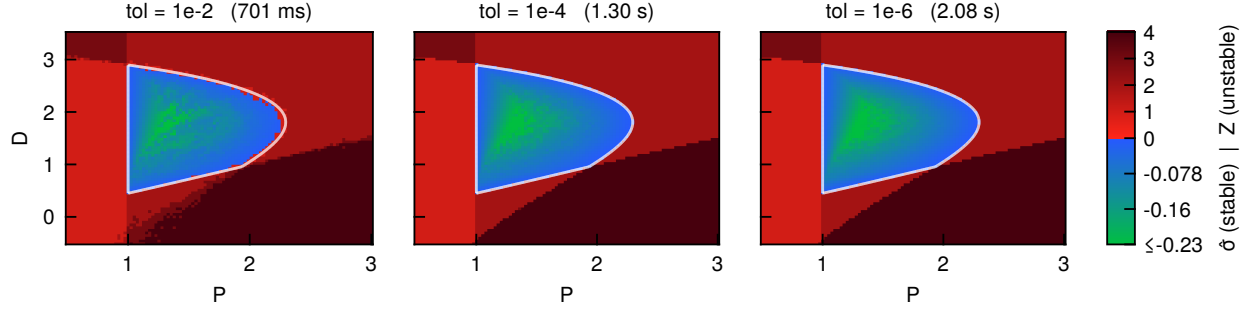


Figure 3: The showcase chart computed with solver tolerances 10^{-2} , 10^{-4} and 10^{-6} (left to right); the wall-clock time of each full sweep is given in the panel titles. The charts are visually near-identical even at 10^{-2} , where the count is already uncertain: the tolerance buys certainty rather than appearance, cf. Fig. 4 and Table 2. The study uses the standard chart window $\omega_{\max} = 10^4$ and the reduced form (20) of the characteristic function.

Table 2: Tolerance study on the showcase system (grid as in Fig. 3): sweep time, integer residual statistics, the number of grid points where the rounded count would be ambiguous ($\varepsilon \geq 1/4$), the number whose Z actually deviates from the 10^{-6} -tolerance reference, and the subset of those that flip the stable/unstable classification. The “uncertain” and “wrong” columns measure genuinely different things: a skipped peak shifts $\tilde{Z}$ by exactly one and leaves it just as close to an integer, so a point can be wrong while looking certain – which is precisely why the cross-check (11) is needed.

tolerance	CPU time	mean ε	max ε	mean/tol	max/tol	uncertain	wrong Z	flips
10^{-2}	701.0 ms	$2.7 \cdot 10^{-2}$	$5.0 \cdot 10^{-1}$	3	50	210	349	89
10^{-4}	1.30 s	$1.8 \cdot 10^{-4}$	$3.9 \cdot 10^{-2}$	2	389	0	1	0
10^{-6}	2.08 s	$1.4 \cdot 10^{-5}$	$1.4 \cdot 10^{-4}$	14	138	0	0	0

6.3. Tolerance as a universal accuracy dial

Figure 3 shows the identical chart computed with tolerances 10^{-2} , 10^{-4} and 10^{-6} , together with the measured sweep times, and Fig. 4 shows the corresponding integer-residual fields (10); the ladder deliberately starts beyond the safe range. Three observations matter for practice. First, the tolerance buys *certainty*, not appearance. The panels are visually near-identical, yet the loosest one is genuinely broken: measured against the 10^{-6} reference, 10^{-2} miscounts 349 of the 8000 points and flips the stable/unstable classification on 89 (Table 2). That failure is anything but silent – the residual field of Fig. 4 lights up across the whole chart, with 210 points beyond the rounding threshold – and one decade into the safe range the problem is essentially gone: at 10^{-4} a single count of the 8000 deviates, by one and without flipping the classification, and at 10^{-6} none. Second, while the integrator dominates the error budget, the integer residual tracks the requested tolerance with a roughly constant factor,

$$\varepsilon \approx 100 \cdot \text{tol} \quad (21)$$

to within an order of magnitude; the factor is inherited from the accepted-step count of the march. At the standard window $\omega_{\max} = 10^4$ the truncated tail of this velocity-feedback system, $\approx 2 \cdot 10^{-5}$, takes over once the tolerance drops

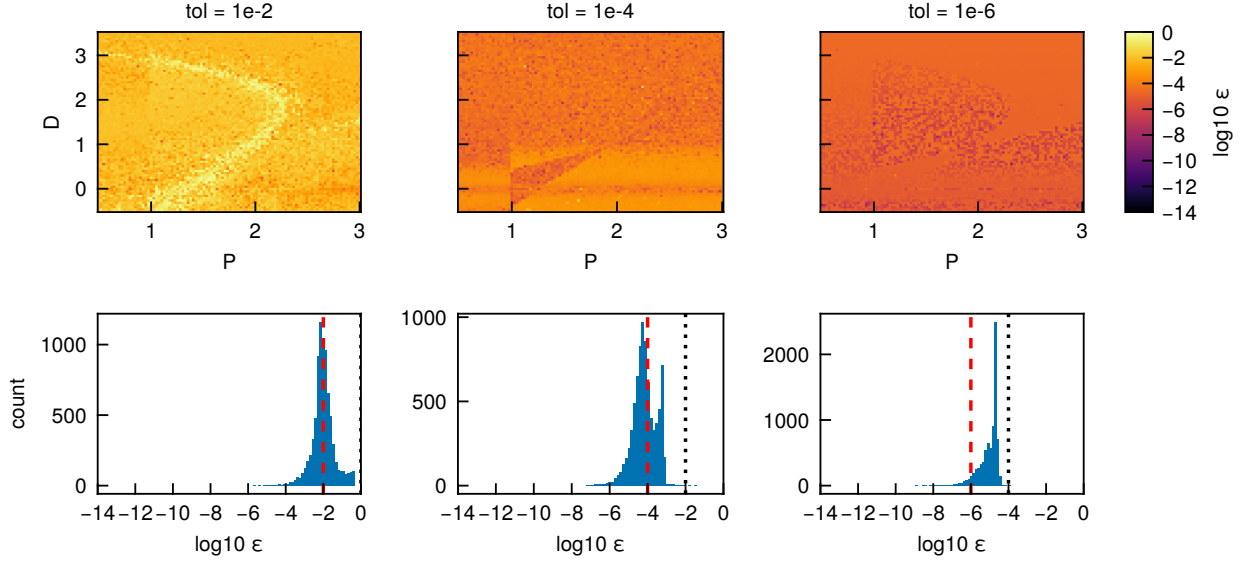


Figure 4: Self-validating error: fields and histograms of the integer residual $\varepsilon = |\tilde{Z} - \text{round}(\tilde{Z})|$, cf. (10), for the three tolerances of Fig. 3. Dashed red line: the requested tolerance; dotted black line: the practical rule $\varepsilon \approx 100 \text{ tol}$. The residual scales with the tolerance uniformly over the parameter plane, amplified by the accepted-step count of the frequency march, until the tail truncated at $\omega_{\max} = 10^4$ (of order $2 \cdot 10^{-5}$) becomes the dominant error.

below $\sim 10^{-6}$ – the mean residual of the tightest panel settles exactly there – and, because every error source surfaces in the same number, the diagnostic itself tells the user that the next decade of certainty must come from $\omega_{\max}$, not from the tolerance. The accuracy of Z can therefore be *prescribed* rather than discovered: a user who wants the count certain to 10^{-3} asks for 10^{-5} , which is exactly the package default. Third, the CPU-time premium of tightening by

For chart work a miscount is a milder error than a displaced boundary, and the two degrade differently: the count is a rounded quantity and fails discontinuously, whereas the tracked $\hat{\sigma}$ inherits the tolerance smoothly. The traced boundary is built from $\hat{\sigma}$ alone through the objective (16), and it stays a small fraction of one grid pixel away from the reference trace at every tolerance studied (data/boundary_shift.csv). The practical recipe follows: a moderately

loose tolerance (10^{-3} – 10^{-4}) is an appropriate tool for an initial search – the residual field will announce if it was too loose – after which the interesting region is recomputed at the default setting to certify the counts, with the cross-check (11) guarding the one error mode near the boundary that the residual cannot reveal.

A flagged pixel need not be recomputed with a different backend: the march can repair itself (peak_repair=true). The skipped peak is easy to *locate* even when it is far too narrow to *integrate*: its center is a minimum of $|\mathcal{D}(\sigma + i\omega)|^2$, which dips toward that minimum over an $O(1)$ frequency range no matter how narrow the peak is. A root-finding callback stops the integrator at every such minimum, recomputes the phase increment of the crossing step by quadrature whose subdivision is forced into the located peak, and restarts the march at the minimum. Measured across the Hopf point of this chart (data/peak_repair.csv): the plain march loses one count from $\Delta P = 10^{-6}$ inward,

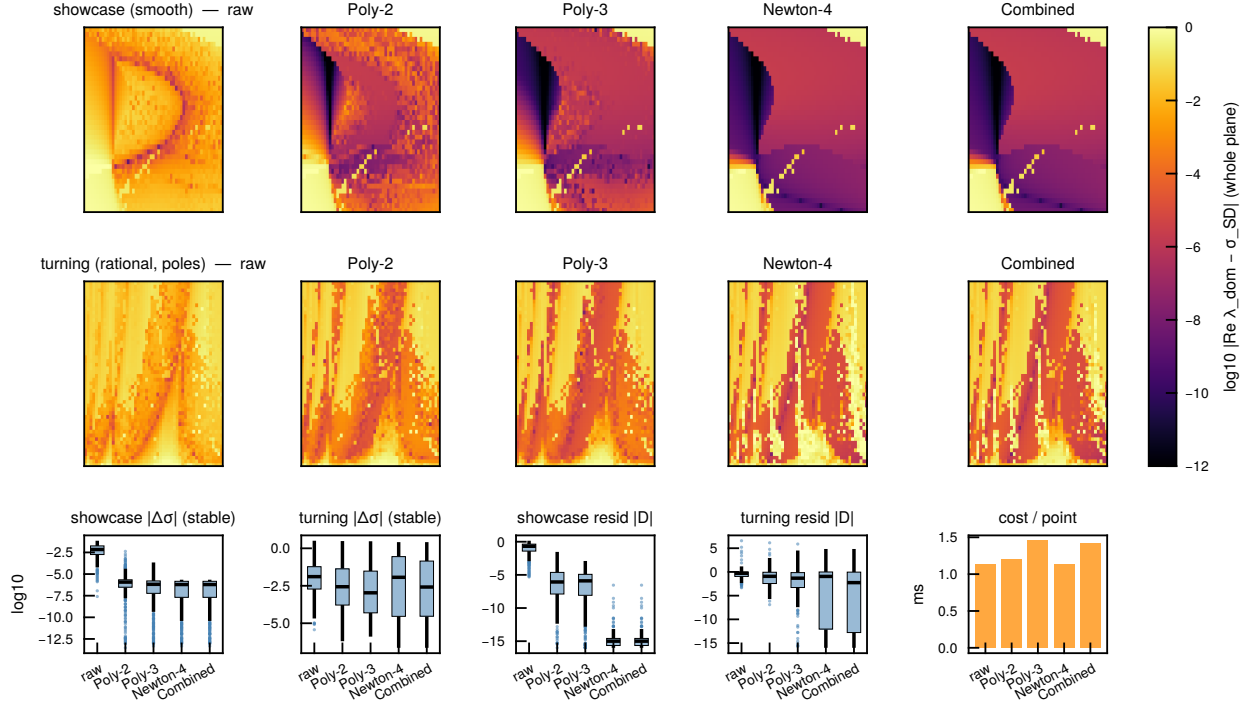


Figure 5: The optional root refinement, judged by method-independent metrics on two systems – a smooth quasi-polynomial (the showcase, top) and a rational frequency-response function with poles (the turning-lobe example of Appendix Appendix A, middle). Error *fields*: per-pixel $\log_{10} |\operatorname{Re} \lambda_{\text{dom}} - \sigma_{\text{SD}}|$, the deviation of the dominant root’s real part from an independent semi-discretization reference, for the raw estimate and the four polishes. Error *bars* (bottom): distributions, over the stable pixels, of the residual $|\mathcal{D}(\lambda_{\text{dom}})|$ (distance to an actual root) and of the same σ -deviation, for each system, together with the per-point cost. The raw estimate is the smoothest field; every polish sharpens the smooth system uniformly, while on the rational system Newton becomes patchy (it can jump to a neighbouring root near a pole) and the Taylor and combined polishes stay consistent.

the Gauss–Kronrod backend – which must *discover* the peak – loses it silently at $\Delta P = 10^{-10}$, whereas the repaired march, which *knows* where the peak is, still counts correctly there. The repair matters only in the immediate vicinity of the boundary, and arming it for a whole sweep costs $\approx 1.9\times$ the plain march, which is why it is off by default; the economical pattern is to sweep plainly, flag with (11), and re-evaluate only the flagged pixels with the repair on.

6.4. Rightmost roots and their refinement

Figure 5 quantifies the rightmost-root estimates with a yardstick that favours no method: the residual $|\mathcal{D}(\lambda)|$, which measures distance to an actual root, and the deviation from an independent semi-discretization solve.¹ Three facts follow. First, the raw tracked value $(\hat{\sigma}, \hat{\omega})$ already places the decay rate to $6.9 \cdot 10^{-3}$ on the smooth system and $1.3 \cdot 10^{-2}$ on the rational one, its sign – all a chart needs – is correct throughout, and being a single closed-form expression per pixel it is also the smoothest field, which is why it is the default. Second, no single polish wins

¹ A natural but circular way to rank the polishes is to measure each against many Newton iterations taken as “truth”, which trivially makes a few Newton steps look best; the metrics used here avoid that trap.

Table 3: Median error of the dominant root’s decay rate $\text{Re } \lambda$ against a semi-discretization reference, for each refinement, on the smooth (showcase) and non-smooth (turning) systems of Fig. 5. Times are per parameter point for the complete solve; overhead is measured against the raw (unrefined) solve.

refinement	$ \Delta\sigma $ showcase	$ \Delta\sigma $ turning	time / point	overhead
raw	$6.9 \cdot 10^{-3}$	$1.3 \cdot 10^{-2}$	1.1 ms	–
Poly-2	$1.2 \cdot 10^{-6}$	$2.7 \cdot 10^{-3}$	1.2 ms	+6%
Poly-3	$6.5 \cdot 10^{-7}$	$1.1 \cdot 10^{-3}$	1.5 ms	+28%
Newton-4	$6.1 \cdot 10^{-7}$	$1.2 \cdot 10^{-2}$	1.1 ms	-1%
Combined	$6.1 \cdot 10^{-7}$	$2.6 \cdot 10^{-3}$	1.4 ms	+25%

everywhere: on the smooth system Newton drives the residual $|\mathcal{D}|$ to machine precision and saturates the accuracy of the semi-discretization reference ($6.1 \cdot 10^{-7}$), but on the rational system, where the plant response has poles near the imaginary axis, a Newton step can settle on a neighbouring root and its error *worsens* to $1.2 \cdot 10^{-2}$ – the patchiness visible in its field – whereas the cubic Taylor polish stays consistent at $1.1 \cdot 10^{-3}$, and the combined mode is the robust compromise ($2.6 \cdot 10^{-3}$) that never speckles. Third, the choice is nearly free: even the combined mode adds only about 25% to the complete solve (Table 3). The practical hierarchy of Section 3.3 follows directly: raw for charts, a polish when a specific root matters, σ -contours (Section 6.5) for a precise decay-rate map. Tracking $N_r > 1$ minima returns several dominant roots at once, which the multi-root recipe of Section 7.4 exploits.

6.5. Decay-rate design: σ -contours

Finally, Fig. 6 demonstrates the γ -stability capability: the traced σ -contours (Section 4.4) enclose progressively smaller regions with guaranteed spectral gaps, and they are consistent with the interpolated $\hat{\sigma}$ background computed from the coarse sweep alone. For control design this map answers directly which gain combinations guarantee a prescribed settling rate – at the cost of one extra boundary tracing per contour.

7. Performance analysis

7.1. Convergence and efficiency of the integrators

Figure 7 and Table 4 quantify the choice of back-end, and three points deserve emphasis. First, all adaptive back-ends track the requested tolerance over the studied range – with the error settling roughly two orders of magnitude above it, the rule of thumb $\varepsilon \approx 100 \text{ tol}$ of Section 3.4 – until they meet a common floor of $\approx 8 \cdot 10^{-7}$ set by the tail truncated at $\omega_{\max} = 10^6$ and the estimated leading order, not by any integrator. Second, the order of the pair matters far more than its family. Across the loose half of the ladder the classical fifth-order pair [65] under-resolves the ω^{-1} ripple of this velocity-feedback system, leaving count errors of 0.2–0.34 – within a hair of the rounding threshold – and even at the package default 10^{-5} it leaves $9 \cdot 10^{-2}$ where the ninth-order default [64] leaves $6 \cdot 10^{-5}$: three orders

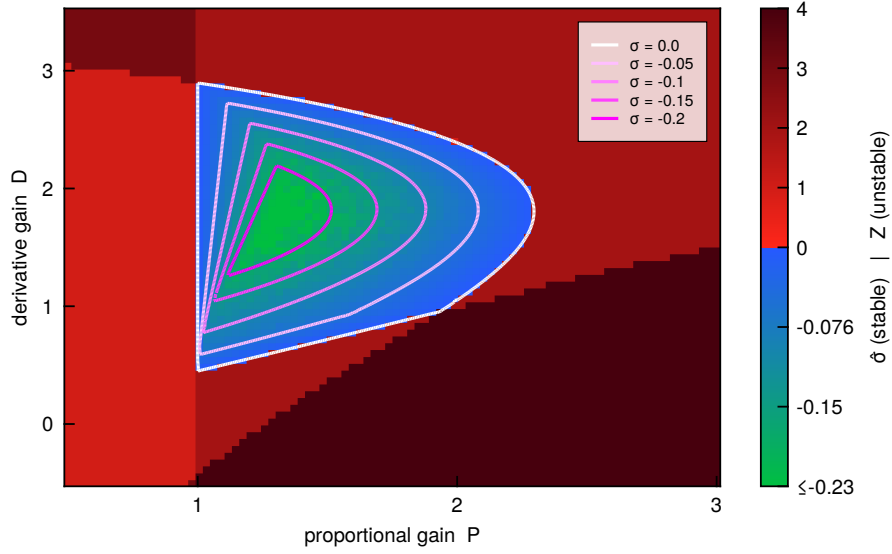


Figure 6: Decay-rate map of the showcase system: colored background – estimated spectral gap $\hat{\sigma}$ from the coarse sweep (stable domain only); curves – guaranteed γ -stability boundaries traced by MDBM for a family of shifted lines $\text{Re } \lambda = \sigma$. The two independent computations delineate the same landscape.

Table 4: Count error $|\tilde{Z} - Z|$ (left block) and single-point CPU time (right block) of the back-ends at selected tolerances; system and point as in Fig. 7.

tol	Vern9 (default)	Tsit5	BS3	Rosenbrock23	QuadGK	Vern9 (default)	Tsit5	BS3	Rosenbrock23
10^{-2}	$1.4 \cdot 10^{-2}$	$2.2 \cdot 10^{-1}$	$5.6 \cdot 10^{-2}$	$2.6 \cdot 10^{-3}$	$3.7 \cdot 10^{-3}$	19.5 ms	0.7 ms	0.2 ms	0.4 ms
10^{-4}	$6.4 \cdot 10^{-3}$	$2.2 \cdot 10^{-1}$	$7.9 \cdot 10^{-2}$	$2.2 \cdot 10^{-2}$	$7.7 \cdot 10^{-7}$	79.9 ms	17.7 ms	9.3 ms	19.9 ms
10^{-6}	$3.9 \cdot 10^{-5}$	$1.8 \cdot 10^{-3}$	$7.0 \cdot 10^{-4}$	$7.5 \cdot 10^{-4}$	$7.6 \cdot 10^{-7}$	139.8 ms	76.6 ms	160.0 ms	367.6 ms
10^{-8}	$7.2 \cdot 10^{-7}$	$5.1 \cdot 10^{-6}$	$2.6 \cdot 10^{-7}$	$6.0 \cdot 10^{-7}$	$7.8 \cdot 10^{-7}$	238.6 ms	223.7 ms	671.3 ms	1.32 s

of magnitude, for a factor of 2.4 in time. The linearly implicit Rosenbrock pair [72, 73, 74] is uniformly dominated, as expected once one recognizes that the problem has no stiffness for it to exploit (Section 3.1); at tight tolerances it needs seconds for a single parameter point where the ninth-order pair needs a quarter of one, which is why the sweep in Fig. 7 stops each method once a point exceeds the cost cap. Third, at the default setting every back-end still rounds to the correct count, so the choice of integrator does not change the chart – what differs, by orders of magnitude, is the *margin* to the rounding threshold, and the default is chosen for that margin rather than for raw speed.

The one situation in which the back-ends genuinely differ is the immediate vicinity of a stability boundary (Table 5). As the crossing root approaches the integration line, the integrand peak narrows in proportion to ΔP (measured half-width $6.6 \cdot 10^{-8}$ at $\Delta P = 10^{-6}$). By $\Delta P = 10^{-8}$ the marching solver steps clean over the peak, loses exactly $\pm\pi$, reports one root too few – and returns a success flag and a perfectly healthy integer residual. Gauss–Kronrod survives the same test, though not by cleverness: the persistent delay ripple forces it to subdivide finely across the whole range

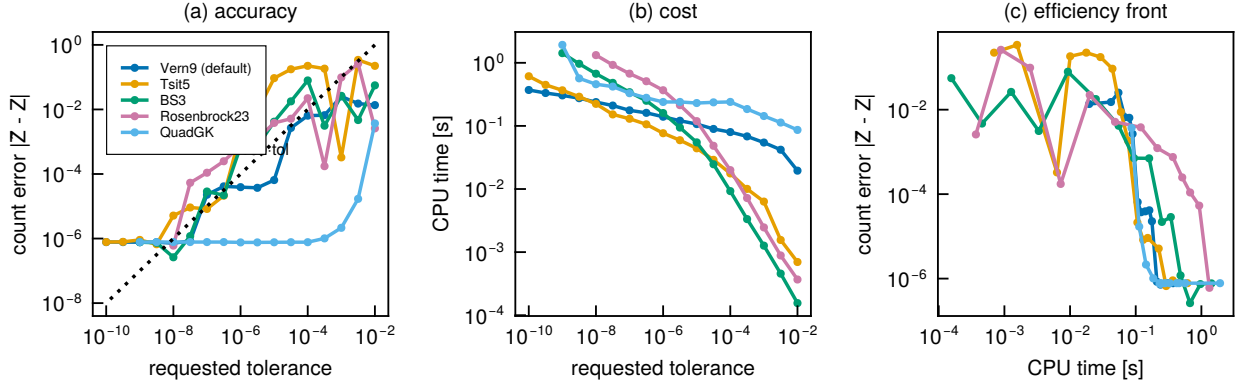


Figure 7: Integration back-ends at an unstable point of the showcase system, where the exact count $Z = 2$ is known (verified independently by a high-resolution semi-discretization eigenvalue count), so the plotted count error $|\tilde{Z} - Z|$ is an *absolute* error measure requiring no reference. (a) Accuracy versus requested tolerance; the dotted line marks the practical rule error $\approx 100 \text{ tol}$. (b) Cost versus tolerance. (c) Efficiency front. Error is on the vertical axis in both (a) and (c). A curve ends where a single parameter point exceeded a cost cap of one second, which is itself informative.

Table 5: Approaching a Hopf boundary of the showcase system at distance ΔP in the gain. The peak half-width equals the true $|\hat{\sigma}|$, proportional to ΔP ; the tracked estimate reproduces it until it saturates at the solver-tolerance floor (last two rows). The marching solver silently loses one root once the peak falls below its step resolution, while the integer residual stays small ($\tilde{Z}$ shifts by exactly one).

ΔP	$\hat{\sigma}$ (peak half-width)	Z march	Z quadrature	ε march	ε quadrature
$1 \cdot 10^{-2}$	$6.6 \cdot 10^{-4}$	4	4	$1.6 \cdot 10^{-5}$	$4.1 \cdot 10^{-7}$
$1 \cdot 10^{-4}$	$6.6 \cdot 10^{-6}$	4	4	$1.6 \cdot 10^{-5}$	$4.1 \cdot 10^{-7}$
$1 \cdot 10^{-6}$	$6.6 \cdot 10^{-8}$	4	4	$1.3 \cdot 10^{-5}$	$4.1 \cdot 10^{-7}$
$1 \cdot 10^{-8}$	$4.3 \cdot 10^{-7}$	3	4	$9.1 \cdot 10^{-6}$	$4.1 \cdot 10^{-7}$
$1 \cdot 10^{-10}$	$3.0 \cdot 10^{-6}$	3	4	$8.3 \cdot 10^{-6}$	$5.2 \cdot 10^{-7}$

anyway, and at the tight tolerance of this stress test (10^{-8}) that refinement still resolves the $\Delta P = 10^{-10}$ peak. At the package default the margin is thinner: there the same experiment (`data/peak_repair.csv`) shows the quadrature losing the count silently at $\Delta P = 10^{-10}$, while the repaired march of Section 6.3 – steered onto the peak by the $|\mathcal{D}|^2$ minimum rather than searching for it – still counts correctly there.

The practical consequences are mild. Landing within $\sim 10^{-8}$ of the boundary is a measure-zero event that a chart grid hits, if ever, in a single pixel, and the damage is a count off by one, not a wild outlier. The MDBM trace is insensitive to it: its objective tends to zero at the boundary regardless of the sign, so a miscount displaces the interpolated boundary point by no more than the skipped strip’s width, $\sim 10^{-8}$ in the gain – far below plotting resolution. And the stable/unstable classification is affected only where the skip flips the count to or from zero – exactly the situation in which the tracked root (still positive, $\hat{\sigma} = +4.3 \cdot 10^{-7}$ at $\Delta P = 10^{-8}$, correctly placing a root right of the axis) disagrees with $Z = 0$ and the sign test (11) fires. Since that test is available only from the ordered

Table 6: Warm chart-level costs (median of repeated runs, 16 threads): full brute-force sweeps for three systems of increasing evaluation cost, with allocated memory. The last column counts the points at which the back-end’s Z deviates from a tight-tolerance reference of the same chart: the adaptive back-ends are required to match everywhere before their time is recorded, while the fixed-step back-end – whose resolution is set by hand – reports its misses honestly.

system	back-end	chart (100×100)	per point [μ s]	alloc [MB]	wrong Z
2nd-order, no delay	ODE Vern9 + root (default)	48.2 ms	5	25	0
2nd-order, no delay	ODE Vern9, Z only	46.0 ms	5	23	0
2nd-order, no delay	QuadGK, Z only	31.8 ms	3	27	0
2nd-order, no delay	fixed-step (500)	36.4 ms	4	1	4324
4th-order + delay	ODE Vern9 + root (default)	426.2 ms	43	25	0
4th-order + delay	ODE Vern9, Z only	418.3 ms	42	23	0
4th-order + delay	QuadGK, Z only	176.2 ms	18	38	0
4th-order + delay	fixed-step (500)	96.3 ms	10	1	4556
2-DOF DAE showcase	ODE Vern9 + root (default)	1.92 s	192	25	0
2-DOF DAE showcase	ODE Vern9, Z only	1.97 s	197	23	0
2-DOF DAE showcase	QuadGK, Z only	2.72 s	272	439	0
2-DOF DAE showcase	fixed-step (500)	80.5 ms	8	1	6489

march, the march remains the default.

7.2. Chart-level cost, memory and cold start

Two operating points matter in practice, and Table 7 separates them. With the *accurate* preset – the standard chart window $\omega_{\max} = 10^4$ and the default $\text{tol} = 10^{-5}$ – a complete 100×100 chart of a scalar characteristic function costs a fraction of a second, and every one of the ten thousand counts matches a tight reference. A *fast* preset ($\text{tol} = 10^{-3}$) buys roughly a factor of two at the price of a few dozen boundary-adjacent miscounts – acceptable for a first look, and flagged by the diagnostics of Section 6.3. Beyond the presets, two user choices dominate the cost on the showcase DDAE, spanning three orders of magnitude together: $\omega_{\max}$ (the velocity feedback’s ω^{-1} ripple makes $\omega_{\max} = 10^6$ cost some sixty times $\omega_{\max} = 10^4$, while buying nothing – the truncated tail is five orders below the rounding threshold) and the form of $\mathcal{D}$ (the automatic extraction costs a factor of 21 per evaluation over the hand-derived expression, Table 1). Choosing badly on both turns the 1.98 s chart of Table 7 into a fifty-minute one; this is what the asymmetric guidance of Section 3.1 means in practice.

Table 6 reports the cost of a *complete* 100×100 chart – ten thousand parameter points – on a AMD Ryzen 7 5800H with Radeon Graphics (16 threads, 15 GB RAM, Julia 1.12.4, package revision b37a016), with the per-point cost listed alongside so that other resolutions can be scaled directly. For scalar characteristic functions a full chart takes tens to hundreds of milliseconds, and the entire hybrid workflow (colored sweep plus MDBM boundary)

Table 7: The two operating points of the method on the same 100×100 chart: the *accurate* preset (the default tolerance) and a *fast* preset for first exploration. The last column counts the points whose Z differs from a tight reference, i.e. the price of the fast preset, which buys roughly a factor of two in time.

system	preset	$\omega_{\max}$	tol	chart (100×100)	per point [μ s]	wrong Z
2nd-order, no delay	accurate	10^4	10^{-5}	47.4 ms	5	0
	fast	10^4	10^{-3}	36.3 ms	4	0
4th-order + delay	accurate	10^4	10^{-5}	437.2 ms	44	0
	fast	10^4	10^{-3}	205.3 ms	21	14
2-DOF DAE showcase	accurate	10^4	10^{-5}	1.98 s	198	0
	fast	10^4	10^{-3}	1.18 s	118	45

Table 8: What the MDBM boundary trace buys: from an initial mesh with the listed refinement levels, the traced boundary has the equivalent resolution of a uniform grid of the given size, at the listed number of function evaluations. The saving factor is the ratio of the two point counts.

system	initial mesh	levels	equiv. resolution	evaluations	boundary pts	time	saving
2nd-order, no delay	20×20	4	320×320	1924	405	32.6 ms	53×
4th-order + delay	20×20	4	320×320	1958	417	113.1 ms	52×
2-DOF DAE showcase	20×20	4	320×320	2969	686	677.7 ms	34×

finishes well inside one second – genuinely interactive. The last column is worth a glance: the fixed-step back-end is the fastest entry for the delayed systems and gets thousands of the ten thousand counts wrong, for the reason given in Section 3.5. The showcase DDAE is listed with its hand-derived characteristic function, as are all figures in this paper; what the automatic extraction of Section 5 would cost instead is measured separately in Table 1. Memory stays modest throughout because the phase ODE carries a single scalar state and the sweep stores only the five result fields; no large matrices are formed at any point. Cold start is a one-time cost dominated by package loading: in a fresh Julia process, loading takes 5–8 s and the first thousand-point sweep 0.07–0.2 s thanks to the precompilation workload shipped with the package, after which every further chart runs at the warm speeds of Table 6.

Table 8 makes the boundary-tracing argument of Section 4.2 concrete. Four refinement levels from a 20×20 mesh yield a boundary resolved as finely as a 320×320 uniform grid would resolve it, while evaluating only the cells that actually bracket the boundary – a saving of 34 to 53 times in function evaluations, which is precisely why the high-resolution boundary is affordable on top of a deliberately coarse colored background. These numbers substantiate the practical claim of the paper: the stability chart of a nontrivial delayed system refreshes fast enough to be explored *interactively*, which changes how such charts are used in design – parameter studies and optimization loops can call the entire chart pipeline as an inner function.

Table 9: Complete 100×100 charts of the three benchmark systems with the integration back-ends and with the discrete phase-unwrapping march of Section 3.6, all measured in one session on one machine (Intel(R) Core(TM) i5-10400 CPU @ 2.90GHz, 12 threads), so that the speed-up factors are free of hardware differences. The integration back-ends use the chart window $\omega_{\max} = 10^4$, the discrete march $\omega_{\max} = 10^5$; the wrong-Z column counts the deviations from a tight-tolerance (10^{-8}) integration reference of the same chart. The turning model enters the discrete march with its denominators cleared (Section 3.6).

system	back-end	chart (100×100)	per point [μ s]	wrong Z	speed-up
4th-order + delay	ODE Vern9 + root (default)	360.1 ms	36.0	0	1.0×
4th-order + delay	QuadGK, Z only	159.5 ms	15.9	0	2.3×
4th-order + delay	fixed-step (500)	75.2 ms	7.5	8075	4.8×
4th-order + delay	unwrap + root (new)	9.6 ms	1.0	0	38×
2-DOF DAE showcase	ODE Vern9 + root (default)	2.35 s	234.5	0	1.0×
2-DOF DAE showcase	QuadGK, Z only	3.44 s	344.5	0	0.7×
2-DOF DAE showcase	fixed-step (500)	92.3 ms	9.2	8863	25.4×
2-DOF DAE showcase	unwrap + root (new)	17.5 ms	1.7	0	134×
two-mode turning	ODE Vern9 + root (default)	10.37 s	1037.2	4	1.0×
two-mode turning	QuadGK, Z only	27.32 s	2731.5	0	0.4×
two-mode turning	fixed-step (500)	91.4 ms	9.1	9903	113.5×
two-mode turning	unwrap + root (new)	34.8 ms	3.5	0	298×

7.3. The discrete march: two orders of magnitude on the CPU

Table 9 measures what Section 3.6 predicts. On the same machine and the same ten thousand points, the discrete march computes every chart without a single wrong count, 38, 134 and 298 times faster than the default ODE back-end, and at 1.0, 1.7 and 3.5μ s per point – a complete 100×100 chart of any of the three systems in 10–35 ms. The gain grows with the difficulty of the system, because the discrete march pays only for what it must resolve: the median point costs 31, 52 and 129 evaluations of $\mathcal{D}$, whereas the integrators pay for every period of the delay ripple up to $\omega_{\max}$. It is even *faster* than the fixed-step trapezoid with 500 steps, which gets 80–99 % of the counts wrong. The quadrature, which skips the root by-product, is no longer competitive anywhere. Near a boundary the discrete march is also the more robust back-end (Table 10): it reproduces the exact count down to $|P - P_b| = 10^{-12}$ on both sides of the Hopf boundary of the showcase system, spending at most 165 evaluations there, while the ODE march at the tight tolerance 10^{-8} misses the narrow peak from $|P - P_b| \leq 10^{-8}$ on – the failure mode discussed above, here resolved by the step rejection on an unpredicted phase jump and by the root-side decision below the resolution of the number format.

The discrete march therefore becomes the recommended back-end for charts of entire characteristic functions, with the ODE march kept for what it does better: the convergence-controlled integer residual of Section 3.4 as an

Table 10: The boundary stress test of Table 5, repeated with the discrete march at its default tolerance (0.3 rad) and with the ODE march at the tight tolerance 10^{-8} . Bold entries are wrong counts. The last column is the number of evaluations of $\mathcal{D}$ the discrete march spends on the point.

$P - P_b$	true Z	ODE Vern9 (10^{-8})	unwrap	evaluations
$-1 \cdot 10^{-2}$	2	2	2	72
$-1 \cdot 10^{-4}$	2	2	2	102
$-1 \cdot 10^{-6}$	2	2	2	118
$-1 \cdot 10^{-8}$	2	2	2	131
$-1 \cdot 10^{-10}$	2	3	2	146
$-1 \cdot 10^{-12}$	2	3	2	165
$+1 \cdot 10^{-2}$	4	4	4	73
$+1 \cdot 10^{-4}$	4	4	4	92
$+1 \cdot 10^{-6}$	4	4	4	106
$+1 \cdot 10^{-8}$	4	3	4	137
$+1 \cdot 10^{-10}$	4	3	4	156
$+1 \cdot 10^{-12}$	4	3	4	157

independent check, and rational $\mathcal{D}$ whose denominators cannot be cleared conveniently. All the timings of Sections 7.1–7.2 were measured with the integration back-ends, on an earlier machine; their *ratios* remain valid, but the absolute chart times of Table 6 are now one to two orders of magnitude above what the package delivers by default.

How the per-point cost of the discrete march divides among CPU and GPU threads, and what that means for the chart time on any hardware, is the subject of Section 8.

7.4. Comparison with semi-discretization

Semi-discretization [9] – here in its efficient library implementation with the multiplication-free variant [25, 26] – approximates the dominant characteristic exponent through the spectral radius of a finite transition matrix, converging polynomially in the number of discretization steps per delay. Table 11 compares it with the present method on the reduced showcase system at a stable, a near-boundary and an unstable point; the dominant root of the present method is obtained by the multi-root recipe of Section 3.2 (ten tracked minima, all refined, maximal real part). That both converge towards the same residual-certified reference validates the two formulations against each other. The substantive difference lies elsewhere: semi-discretization requires the user to choose the number of steps per delay *by hand*, and an insufficient resolution returns a confidently wrong exponent with no warning, whereas the present method sets its own frequency steps, exposes the accuracy through the requested tolerance, and ships two independent consistency checks (the integer residual and the sign agreement (11)). The comparison is also intentionally asymmetric in what the methods deliver: semi-discretization returns the full dominant multiplier cluster and extends unchanged to

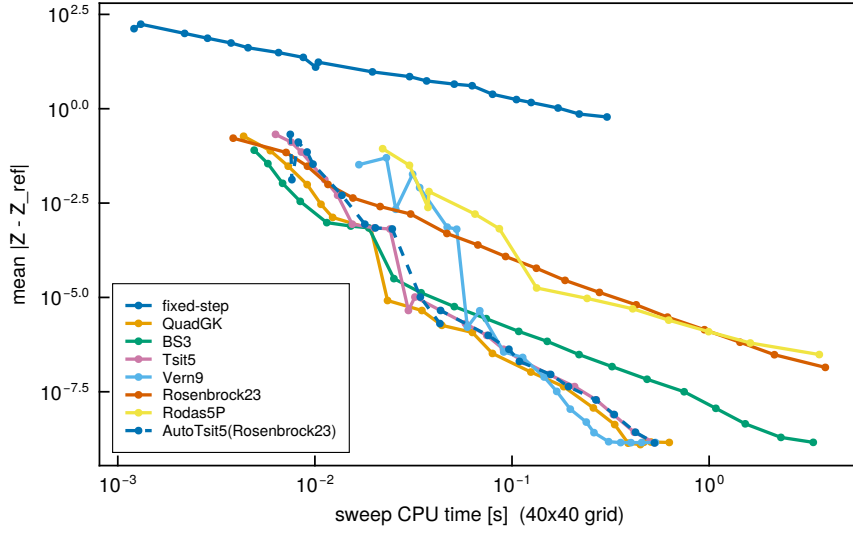


Figure 8: Error-cost Pareto fronts of eight integration back-ends on a 40×40 sweep of the fourth-order benchmark oscillator (Appendix Appendix A): fixed-step trapezoid (step-count sweep), adaptive Gauss–Kronrod, and six ODE solvers (tolerance sweeps).

time-periodic systems, where the argument-principle formulation does not apply; the present method returns exactly what an *autonomous* stability chart needs – the root count, the boundary, and the dominant root on demand – at interactive speed.

7.5. A broader solver comparison

Table 12 isolates what actually drives the cost. The decisive quantity is not the back-end but the decay rate of the phase ripple, set by how far the delayed terms sit below the leading power of $\mathcal{D}$. Where the ripple decays fast (delayed *position* feedback, $\theta' \sim \omega^{-3}$), quadrature recognizes the negligible tail panel by panel and finishes about 2.5 times cheaper than the march; where it decays slowly (delayed *velocity* feedback, $\theta' \sim \omega^{-1}$), the ranking reverses and the march is some three times cheaper. Neither back-end dominates, and the lesson for a user is to spend attention on $\omega_{\max}$ and on the structure of the feedback, not on the choice of integrator. One implementation detail underpins the quadrature numbers: the Gauss–Kronrod back-end is seeded with logarithmically spaced interior breakpoints, one per two frequency decades. Without them the first $G_7\text{--}K_{15}$ panel spans the entire window, its lowest node lands above the whole resonance region for $\omega_{\max} = 10^6$, and the routine returns $\tilde{Z} = n/2$ having never seen a single root – with an integer residual of *exactly* zero, since nothing about the quadrature failed. The breakpoints cost a few dozen extra evaluations and remove the failure mode entirely.

Figure 8 extends the comparison to a wider selection of ODE solvers on the fourth-order benchmark system. The fixed-step trapezoid is *dominated everywhere*: even at $3 \cdot 10^4$ steps – more than an order of magnitude slower than the adaptive default – its worst error over the chart remains several roots, because a uniform grid cannot resolve a peak that narrows without bound as a root approaches the line. All adaptive methods follow their tolerance; higher-order

710 pairs dominate at tight tolerances, and the linearly implicit pair is uniformly dominated – there is no stiffness for it to exploit. The spread between the *adaptive* methods is far smaller than the gap that separates all of them from a fixed grid, which is the practical message of this section.

8. Parallel execution: from one core to a graphics processor

715 The absolute chart times quoted so far depend on the hardware they were measured on, and a reader with a different machine cannot transfer them. A more useful description separates two numbers: the cost of one parameter point on one thread, and how that cost divides among many threads. This section measures both, on CPUs and GPUs, and shows that the second is almost ideal – which is what makes real-time charts possible, and what lets anyone predict the chart time on their own hardware.

8.1. Why the sweep parallelizes perfectly

720 A stability chart is an embarrassingly parallel computation: the parameter points are independent, each needs only the user’s characteristic function and the march of Section 3.6, and nothing is exchanged between them. The discrete march adds three properties that a parallel machine rewards: its state is a fixed handful of scalars (no allocation, no solver object), every step is the same short sequence of operations (one evaluation of $\mathcal{D}$ and $\mathcal{D}'$ by dual numbers, a comparison, a step update), and its cost per point varies only moderately across a chart – the median point of the three
725 benchmark systems needs 31, 52 and 129 evaluations (Table 9), and the maximum is a small multiple of the median. The same Julia source therefore runs as a multithreaded CPU loop and as a GPU kernel with one thread per point; we implemented it once with `KernelAbstractions.jl`, which compiles it for both.

8.2. Measured scaling

Figure 9 shows the time of a chart of 10^6 points as a function of the number of threads, and Table 13 extracts the
730 characteristic numbers. On the CPU the result is what the independence of the points promises. One thread needs $3.8\mu\text{s}$ per point; every doubling of the threads halves the time, so that 16 threads are 15.0 times faster and 48 threads 38 times – the small deficit coming from hyper-threading, where two threads share one core. The six-core desktop follows the same line up to its six cores (5.6 times) and gains another factor of 1.6 from its twelve hardware threads. A chart that takes 3.8 s on one core (10^6 points of the fourth-order benchmark) thus takes 0.1 s on a 48-thread server.

735 On the GPU the same picture holds, shifted by a constant factor. A single GPU thread is slower than a CPU thread – $21\text{--}32\mu\text{s}$ per point in Float32, five to eight times the CPU time, because a GPU core runs at a lower clock and executes one instruction at a time without the speculation and large caches of a CPU core. From there the time again falls in proportion to the number of threads, over four decades, until the device is saturated. Two short deviations from the ideal line are instructive. Below 32 threads the gain is sublinear, because a GPU executes threads in groups of 32 (a *warp*) in lockstep: the threads of one warp advance together, and a warp is only as fast as its slowest point.
740 Between 64 and 256 threads the curve flattens, because a single thread block runs on one multiprocessor; the ideal

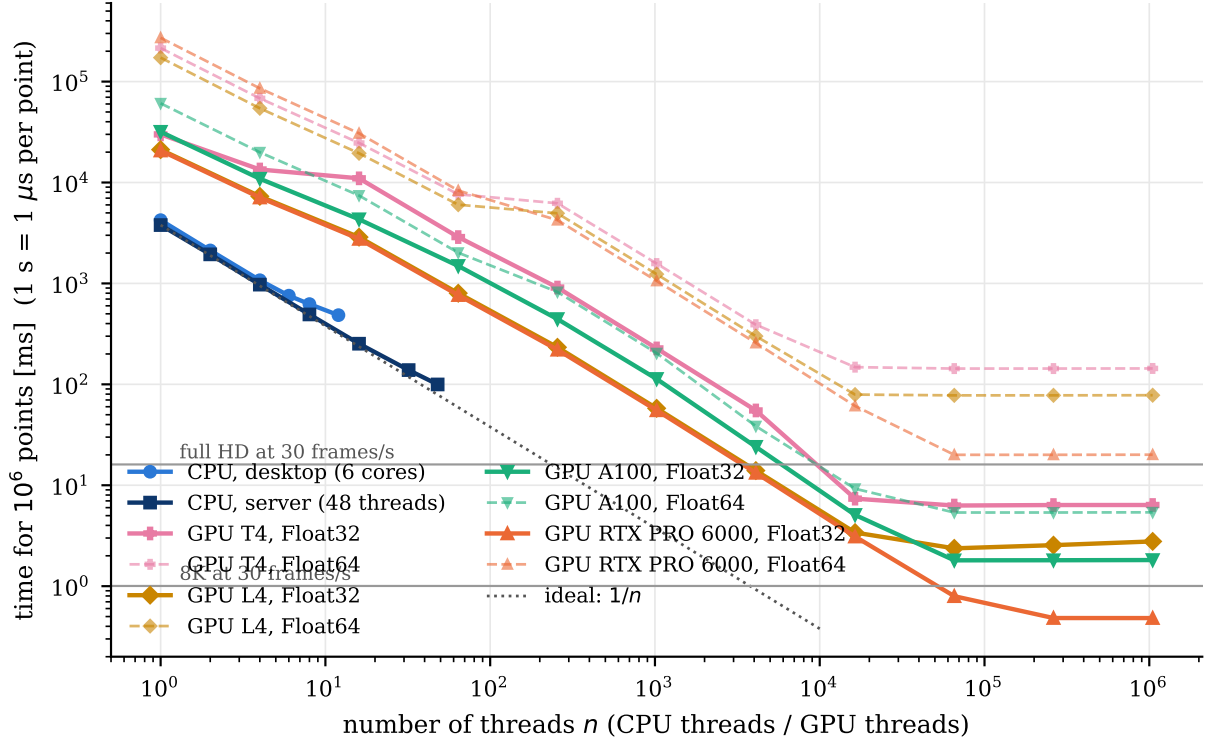


Figure 9: Parallel scaling of the discrete march on the fourth-order benchmark (Appendix Appendix A): time for 10^6 parameter points – a 1000×1000 chart, so that 1 s corresponds to $1 \mu\text{s}$ per point – versus the number of threads n , on two CPUs (Float64; a six-core desktop and a 48-thread server) and on four GPUs (Google Colab), where exactly n GPU threads march the chart (solid: Float32, dashed: Float64). The dotted line is the ideal $1/n$ through the single-thread CPU time; the horizontal lines mark the per-point cost at which a full-HD or an 8K chart is computed thirty times per second.

slope resumes as soon as the blocks spread over the device. Beyond some 10^4 – 10^5 threads every multiprocessor is busy and the curves level off at the saturated cost of each card.

8.3. A performance model for any hardware

The measurements condense into a model with two hardware numbers. If one thread needs t_1 per point and the device sustains P_{eff} concurrent marches, a chart of N points on n threads takes

$$T(N, n) \approx \frac{N t_1}{\min\{n, P_{\text{eff}}\}}. \quad (22)$$

On a CPU, P_{eff} is the number of cores (plus a fraction for hardware threads). On the four GPUs it is 1.2 to 2.6 times the number of CUDA cores in Float32 (Table 13): a GPU core can keep more than one march in flight because it switches between warps while one waits for memory or for the result of a long instruction. Since t_1 in Float32 differs by less than a factor of two between the four cards, the saturated chart time is set almost entirely by the number of cores – the chart is compute-bound, not memory-bound, which is expected for a method whose state fits in registers.

The model thus predicts the time on an unlisted card from its core count and the per-point cost measured once on any GPU, within a factor of about two; clock rate, register pressure (which limits the number of resident warps) and the per-point variation of the step count account for the remainder. The per-point cost itself scales with the cost of one evaluation of $\mathcal{D}$ and with the number of evaluations: the two-mode turning model, with four times the evaluations of the fourth-order benchmark, saturates the RTX PRO 6000 at 3.0 ns instead of 0.48 ns per point.

Float64 follows the same model with a larger t_1 , and Fig. 9 shows the consequence of the hardware design. Graphics-oriented cards (T4, L4, RTX PRO 6000) execute double-precision operations at 1/32–1/64 of the single-precision rate, so their saturated Float64 time is 23–41 times the Float32 time, whereas the A100, built for scientific computing, pays a factor of three. Since Float32 gives the same chart as Float64 point for point (Table 14), Float64 serves as a reference on the GPU rather than as a production format.

8.4. Real-time charts at 8K resolution

The horizontal lines of Fig. 9 translate the per-point cost into frame rates. A chart refreshed thirty times per second leaves 33 ms per frame; for a full-HD chart ($2.07 \cdot 10^6$ points) this needs 16 ns per point, and for an 8K chart ($7680 \times 4320 \approx 3.3 \cdot 10^7$ points) 1 ns per point. A single CPU core is more than two orders of magnitude above the first line, and even the 48-thread server is a factor of six above it (a full-HD chart in 0.2 s, i.e. five frames per second, or real time at a quarter of the resolution). Every one of the four GPUs is below it, and the fastest card, at 0.48 ns per point – 7 841 times the throughput of one CPU core, an effective parallelism of 42 953 – is below the 8K line as well: a full-HD chart takes 1.0 ms and an 8K chart 16 ms. The chart can therefore be recomputed *while a slider is being moved*: the delay, a gain or a damping ratio is changed, and the complete chart – here more than thirty million stability decisions with the rightmost root of each – follows the motion of the hand without visible delay. Table 14 gives the complete full-HD timings with the production settings; measured at 8K on the RTX PRO 6000, the kernel takes $[t]$ ms in Float32 and $[t]$ ms with the Float16 evaluation and the Float32 re-check described below, which yields the same chart. A complete interactive frame – kernel, colouring and transfer of the display image – adds a few milliseconds (gpu/results/interactive_G4.csv).

It must be stressed that these frame rates belong to the given problem. The cost per point is proportional to the cost of one evaluation of $\mathcal{D}$ and to the number of evaluations, so a model whose $\mathcal{D}$ is a large determinant or whose phase ripple decays slowly (Table 12) moves all curves of Fig. 9 upwards by the corresponding factor – the two-mode turning model, for instance, runs at about ten frames per second at 8K and above 150 frames per second at full HD on the same card. What does not change is the slope: whatever the characteristic function, the chart time divides by the number of threads, from a single core to tens of thousands of GPU cores.

Narrow number formats.. Since the march needs only the *phase* of $\mathcal{D}$ to a fraction of a radian, the characteristic function can even be evaluated in half precision (Table 15). Float16 is accurate wherever no root is close to the line: after rescaling the frequency into its range it computes the chart up to several times faster than Float32 on cards with

785 a fast half-precision path, with 0.0169 % wrong counts that are *all* flagged, and 1.756 % flagged points in total. Re-checking the flagged points in Float32 restores the exact chart, and done in place on the device – gather the flagged points, march them in Float32, scatter the results back – the second pass is cheap. The gain depends on the system: for the lightly damped two-mode turning model a third of the points are flagged in Float16, and the hybrid is barely faster than Float32. BFloat16, with only seven mantissa bits, flags 14 % of the chart, and the 8-bit and 4-bit formats are
790 useless for this purpose: with two or three mantissa bits even the *parameters* of a chart are quantized to a handful of distinct values per octave, so that 95–100 % of the points are flagged and up to one in ten counts is wrong; Float4 gets every count wrong. Load balancing, finally, turned out to be unnecessary: a dynamic work queue, in which threads that finish early fetch new points, was not faster than one thread per point on any card, because the cost per point varies only moderately and the hardware scheduler absorbs the difference.

795 9. Conclusions and outlook

The Nyquist/argument-principle criterion for linear autonomous time-delay systems was reformulated as a scalar ordinary differential equation for the phase of the characteristic function and marched by a high-order adaptive integrator with an exact, automatically differentiated integrand. The outcome is a genuinely fast tool, fast enough for interactive exploration and for use inside optimization loops. The reformulation tames the practically unbounded
800 frequency range and the near-singular integrand through the mature step-size control of modern ODE software, and it makes the accuracy self-validating through the integer residual of the computed winding number, a dimensionless error measure that scales with the requested tolerance – empirically as $\varepsilon \approx 100 \text{ tol}$ – so a single tolerance transfers between systems with no problem-specific tuning. It should be said plainly that the phase ODE carries no stiffness: its right-hand side is independent of the state, so the formulation is a quadrature in disguise, and adaptive quadrature
805 is fully competitive for the count alone. What the ordered march uniquely provides – and what every chart in this paper is built from – is the characteristic root obtained along the way: a smooth first-order estimate at negligible cost, sharpened toward machine precision by an optional Taylor, Newton or combined polish when an accurate root is wanted.

Taking the observation that the count needs only the end value of the phase one step further removed the integration
810 altogether. The discrete phase-unwrapping march sums the exact phase increments between adaptively placed samples and uses the exact phase slope, delivered by the same automatic derivatives, for an ODE-style step control that rejects every step hiding a root transition; transitions below the resolution of the number format are decided by the side of the tracked root. With one evaluation per step and a few dozen evaluations per point it computes the benchmark charts 38–298 times faster than the phase ODE – 10–35 ms for a 100×100 chart on a desktop processor – without a single
815 wrong count, and it resolves the Hopf boundary down to a parameter distance of 10^{-12} . Its fixed-size loop of identical scalar steps is equally at home on a GPU, and the cost of a chart is best described in two hardware numbers: about $3.8 \mu\text{s}$ per point on one CPU thread ($21\text{--}32 \mu\text{s}$ on one GPU thread) for the fourth-order benchmark, divided by the

number of threads up to the effective parallelism of the device – linearly on CPUs up to 48 threads and on GPUs over four decades of thread counts. The fastest of the four GPUs measured thus computes a full-HD chart in about a millisecond and an 8K chart fast enough to follow a moving slider, and the same model predicts the chart time on any other hardware from its core count. Float32 gives the Float64 chart point for point; the characteristic function can even be evaluated in Float16, whose rare errors are all flagged, while 8-bit and 4-bit formats are useless for stability charts. The ODE march remains the reference with the convergence-controlled integer residual; the discrete march is the tool for charts, real-time exploration and parameter optimization.

The root estimate transforms the integer-valued root-counting map into an interpolable chart: one coloring simultaneously shows the number of unstable roots and the spectral gap of the stable domain; the estimated root distance serves as a scalar objective for the multi-dimensional bisection method, which traces the true stability boundary (rather than all root-crossing D-curves) at high equivalent resolution; the most robust parameter point follows from the largest inscribed circle; and shifted integration lines yield guaranteed decay-rate (γ -stability) contours. The characteristic function itself is extracted algorithmically from a simulation-ready right-hand side – including nonlinear models (linearized automatically) and singular mass matrices – so the complete workflow requires nothing beyond the model and the parameter ranges. The case studies demonstrate the identical code path on retarded, neutral, distributed-delay, differential–algebraic, transcendental, high-dimensional and fractional-order systems, at chart-generation speeds that allow interactive exploration and optimization.

One modelling lesson deserves to be carried away from the case studies, because it costs nothing and outweighs any tuning of the numerics: where a model admits a natural open-loop/feedback split, the characteristic function should be written as the return difference $1 + L(\lambda)$ rather than as a raw determinant. The zeros are the same, the added poles are the open-loop roots and lie in the left half-plane for an open-loop stable structure, and the leading order becomes exactly zero – which removes both the need to estimate it and the astronomical magnitudes that a high-order determinant produces.

The limitations delineate the outlook. First, the method is restricted to autonomous (time-invariant) linear systems: for time-periodic coefficients the characteristic function would have to be replaced by a Hill-type infinite determinant, and time-domain discretization methods such as semi-discretization [9, 26] remain the more practical choice there. Second, away from the stability boundary the rightmost-root estimate is of first order unless refined, so the method complements rather than replaces full spectral solvers when the deep spectrum is needed. Third – intrinsic to the argument principle itself – a peak that no sample lands on shifts the count by one while leaving it an integer, so the residual cannot see it; the cross-check between the count and the independently tracked root detects this at no cost, the optional detect-and-repair callback of Section 6.3 mends the flagged pixels within the march, and the discrete march rejects such steps by construction – its own blind spot being a pair of transitions in the same direction within one step, which an entire $\mathcal{D}$ and a band-limited step cap remove. A formally guaranteed count would still require a different device, such as interval arithmetic. Fourth, for neutral systems beyond the essential instability threshold the root count diverges with the truncation frequency; the classification into stable and unstable domains nevertheless remains

correct (Appendix Appendix A). Fifth, the fractional-order results are preliminary: the counting on the principal branch was validated numerically – including the reproduction of a published fractional-controller design chart – but a rigorous treatment of branch-cut effects, in particular for shifted integration lines, is left for future work. Finally, for large models the cost is dominated by each evaluation of $\mathcal{D}$ rather than by the method; where the feedback has low rank the matrix determinant lemma already removes most of that cost (A.3), and caching factorizations across frequencies, extending the automatic extraction to neutral right-hand sides, evaluating large determinants on the GPU, and coupling the real-time charts to formal optimization loops and to a browser front end are natural continuations.

Software and reproducibility

The method is implemented in the open-source Julia package `InterpolatedNyquist.jl` (MIT license, <https://github.com/bachrathyd/InterpolatedNyquist.jl>), built on `OrdinaryDiffEq.jl` [66], `ForwardDiff.jl` [67], `QuadGK.jl` [71], `MDBM.jl` [51] and `DelaunayTriangulation.jl` [75]. The GPU implementation of the discrete march (NyquistGPU, with a Google Colab notebook) is in the `gpu/` directory of the same repository. All figures and tables of this paper can be regenerated by the scripts in the `paper/` directory of the repository; the benchmark hardware and software versions are embedded automatically in the manuscript.

Funding

The research leading to these results was supported by the Hungarian Scientific Research Fund (NKFIH-AG-152125).

Declaration of competing interest

The author declares that there is no known competing financial interest or personal relationship that could have appeared to influence the work reported in this paper.

Appendix A. Case-study gallery

This appendix demonstrates the identical workflow of Section 4.5 – coarse colored sweep, MDBM boundary, combined coloring (15) – on eleven further systems covering all supported model classes, plus the reproduction of a published design chart. The complete scripts, grid resolutions and timings are part of the package repository.

These charts are the speed argument.. Sections 6 and 7 make the accuracy case, with tolerances down to 10^{-8} . The gallery answers the complementary question – *what does a chart cost when one simply wants to see the picture?* – and so every panel is computed at settings chosen for speed: a 100×100 background grid and a tolerance of 10^{-4} , one decade looser than the package default. At these settings the count of a few boundary-adjacent pixels may be off

by one (Section 6.3); that is precisely the trade a first exploration should make, it is invisible at chart resolution, and the traced boundary – built from $\hat{\sigma}$, not from the count – is unaffected. The MDBM traces start from a 7×7 mesh and refine until the boundary carries roughly four times the resolution of the background grid. $\omega_{\max} = 10^4$, the standard window of this paper, is used wherever it helps; five panels use less, for reasons of substance (the ripple of a neutral system, A.4–A.6, does not decay at all, so a wider window buys nothing, and for A.8–A.9 the Kelvin–Voigt damping extinguishes the integrand long before $\omega = 200$). Each panel reports the measured integer residual of its own grid (data/gallery_timings.csv): on every retarded panel the median residual is $\sim 10^{-4}$, while on the neutral panels it is ≈ 0.1 – the irreducible tail oscillation of Section 3.4, not a symptom of truncation.

Reading the charts.. Every panel uses the combined field C of (15): outside the stable domain the colour encodes the integer number of unstable roots as flat plateaus; inside, the same colour axis continues below zero and encodes the estimated decay rate $\hat{\sigma} < 0$, varying smoothly. One map therefore answers three questions at once – where the system is stable, how robustly, and how many roots must be stabilized elsewhere. Following Section 3.2, the shading uses the *dominant* root (five tracked minima, all refined, maximal real part) rather than the single closest minimum, and the same convention keeps the MDBM objective continuous across branch crossings. The black curve is the boundary traced by MDBM from (16), not a contour of the coloured grid, and each panel is annotated with the wall-clock cost of both stages.

A.1 Fourth-order delayed oscillator.. $\mathcal{D}(\lambda) = c_1 \lambda^4 + \lambda^2 + 2\zeta\lambda + 1 + (P + D\lambda)e^{-\lambda\tau}$ with $c_1 = 0.03$, $\zeta = 0.02$, $\tau = 0.5$, swept over the control gains (P, D) . This is the regression benchmark of the package (test suite and performance history) and the system of the solver comparison in Fig. 8.

A.2 Delayed oscillator.. $\mathcal{D}(\lambda) = \lambda^2 + a\lambda + b e^{-\lambda/2}$ over (a, b) : the textbook retarded case with both divergence and Hopf boundaries.

A.3 Distributed delay.. $\mathcal{D}(\lambda) = \lambda^2 + a\lambda + b(1 - e^{-\lambda\tau})/\lambda$, $\tau = 1$, i.e., a uniform delay kernel $\int_0^\tau x(t - \vartheta)d\vartheta$. The kernel integral appears in closed form in $\mathcal{D}$; the removable singularity at $\lambda = 0$ is handled by the series fallback in the model code and by the origin offset of Section 3.1.

A.4–A.5 Neutral systems.. $\mathcal{D}(\lambda) = \lambda^2 + a\lambda^2 e^{-\lambda} + 1 + c e^{-\lambda}$ and its high-gain damped variant $\mathcal{D}(\lambda) = \lambda^2 + a\lambda^2 e^{-\lambda} + 5\lambda + c e^{-\lambda}$, swept over (a, c) with $|a| < 1$. The delayed acceleration term makes the equations neutral: the phase integrand oscillates persistently up to arbitrarily high frequencies, so the truncation is kept moderate ($\omega_{\max} = 200$) – the resulting tail error is bounded by $\arcsin |a|/\pi < 1/2$ and never corrupts the rounded count – and the leading-order fit (7) delivers the effective order $n = 2$ where the naive point estimator diverges.

A.6 PDA control and essential instability.. Acceleration feedback is popular in active vibration control, but the delayed acceleration term $A\ddot{x}(t - \tau)$ turns the closed loop into a neutral system with characteristic function

$$\mathcal{D}(\lambda) = \lambda^2 + 2\zeta\lambda + 1 + (P + D\lambda + A\lambda^2)e^{-\lambda\tau},$$

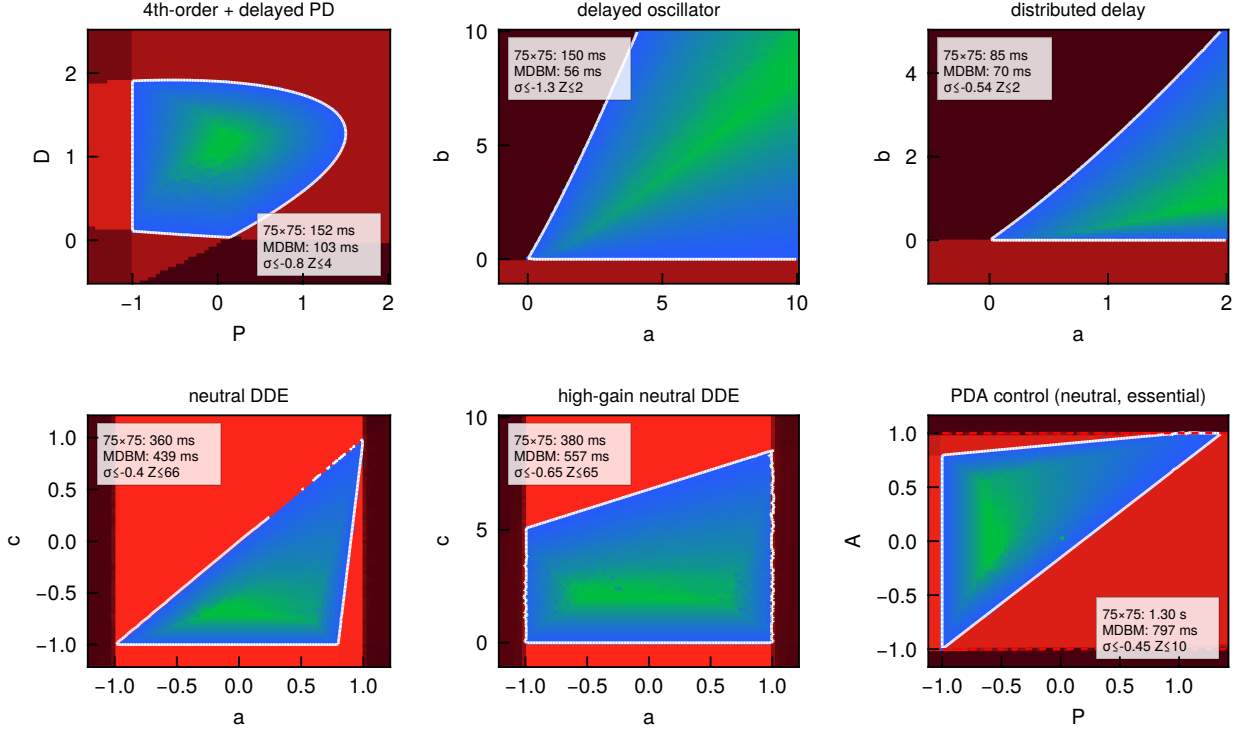


Figure A.10: Case studies, part I: (top row) fourth-order delayed oscillator, delayed oscillator, distributed delay; (bottom row) neutral DDE, high-gain neutral DDE, and PDA control with essential instability (the analytic essential boundaries $|A| = 1$ are dashed red).

910 whose neutral coefficient is $1 + Ae^{-\lambda\tau}$. For $|A| > 1$ the essential spectrum moves to $\text{Re } \lambda = \ln |A|/\tau > 0$: infinitely many roots become unstable and $Z \rightarrow \infty$. With a finite $\omega_{\max}$ the computed count saturates at a large value ($\sim \omega_{\max} \tau / (2\pi)$), so the method cannot – and need not – report the exact count there; crucially, the *classification* into stable and unstable domains remains correct, and the chart in Fig. A.10 displays both the regular Hopf-type lobes and the analytic essential boundaries $|A| = 1$, which the MDBM trace locates without special treatment. The count is capped in the plot for
915 readability. This case study substantiates the claim of Section 9 that essential loss of stability is detected robustly even though the count itself diverges.

A.7 Multi-mode turning lobes.. The regenerative turning model with a two-mode flexible structure, $\mathcal{D}(\lambda) = 1 + w(1 - e^{-2\pi\lambda/\Omega}) \sum_{k=1}^2 g_k / (\lambda^2 + 2\zeta_k \omega_k \lambda + \omega_k^2)$, over spindle speed Ω and chip width w [1, 3, 52]. Two features deserve emphasis. First, $\mathcal{D}$ is *rational* – its structural poles lie in the open left half-plane, so the argument-principle count is unaffected, and $\mathcal{D} \rightarrow 1$ at high frequency gives an effective order $n \approx 0$; no polynomial form is ever constructed. Second, the chart makes the introductory argument visible: towards low spindle speeds the lobes accumulate, and a D-curve-based method would trace an unbounded family of curves of which almost none bounds the stable region, whereas the boundary objective (16) follows only the true stability limit [54].
920

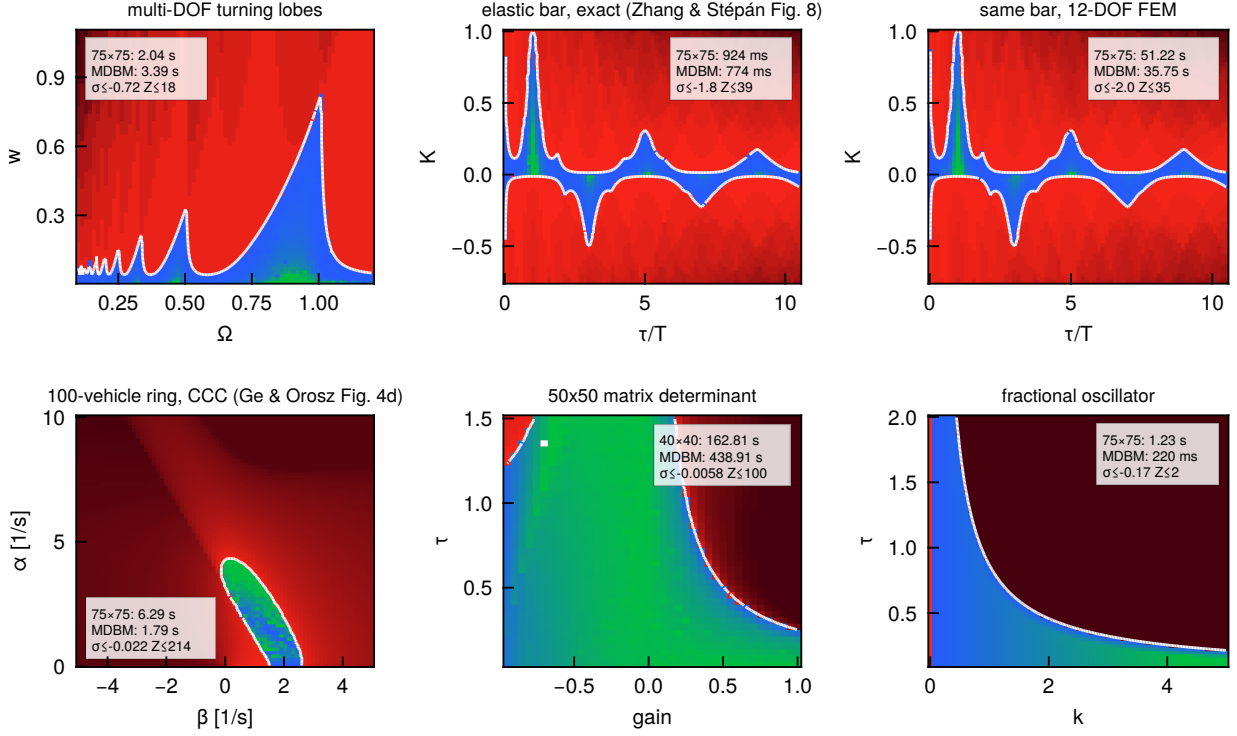


Figure A.11: Case studies, part II: (top row) multi-mode turning lobes, transcendental beam, 12-DOF finite element bar; (bottom row) 50×50 matrix determinant, fractional-order oscillator.

A.8 Transcendental beam, and why the damping model decides everything.. Longitudinal vibration of an elastic bar with delayed boundary feedback gives – *without any spatial discretization* – the exact transcendental return difference

$$\mathcal{D}(\lambda) = 1 + K_p e^{-\lambda\tau} \operatorname{sech}(\gamma(\lambda)L), \quad L = 1, \quad (\text{A.1})$$

cf. the exact analysis of Zhang and Stépán [69] and the finite-DOF hierarchy of Kidd and Stépán [76]. Whether a finite count even exists is decided entirely by the damping model. With external (viscous) damping, $\gamma = \sqrt{\lambda^2 + c\lambda}$, every mode is damped equally: the closed-loop roots accumulate along a vertical line, so the number of unstable roots is 0 or ∞ – the essential-spectrum obstruction of a neutral system (A.6). Real materials, however, dissipate through *rate-dependent* (Kelvin–Voigt) internal damping, $\gamma = \lambda/\sqrt{1+c\lambda}$: the high modes are damped ever harder, only finitely many roots sit near the axis, and $|\operatorname{sech}(\gamma L)|$ decays like $e^{-\sqrt{\omega/2c}}$. The chart uses $c = 0.02$ – a light damping that leaves the low modes and the chart shape essentially unaffected, yet is what makes the count exist at all. The chart over (K_p, τ) in Fig. A.11 shows the resulting stable bands; the example doubles as a reminder that the right damping law is a modelling decision the method faithfully reflects rather than papers over.

A.9 Finite element bar (12 DOF), and the return difference.. The same bar discretized by 12 linear finite elements, with stiffness- proportional (Kelvin–Voigt) damping $\mathbf{C} = \eta\mathbf{K}$. Writing the delayed boundary feedback as the rank-one

matrix $\mathbf{F}(\lambda) = c(\lambda) \mathbf{e}_1 \mathbf{e}_N^\top$, where the scalar $c(\lambda) = -Kh^{-1}(1 + \eta\lambda) e^{-\lambda\tau}$ carries the damped modulus and the clamped-end strain sign (h = element length, $N = 12$), the naive formulation is the raw determinant $\det(\lambda^2 \mathbf{M} + \lambda \mathbf{C} + \mathbf{K} + \mathbf{F}(\lambda))$, and it is a poor one: it grows like λ^{24} , reaches $\sim 10^{39}$ at the top of the integration range, and forces the counting formula
 940 onto the estimate of a large leading order – which leaves an integer residual of 0.12. Factoring out the open-loop dynamics,

$$\mathcal{D}(\lambda) = \det(\mathbf{I} + \mathbf{Q}_0(\lambda)^{-1} \mathbf{F}(\lambda)), \quad \mathbf{Q}_0 = \lambda^2 \mathbf{M} + \lambda \mathbf{C} + \mathbf{K}, \quad (\text{A.2})$$

leaves the zeros untouched, adds poles only at the open-loop roots – all in the left half-plane, so the right-half-plane count is unaffected – and yields $\mathcal{D} \rightarrow 1$ at infinity, hence $n = 0$ exactly and a residual of $2.0 \cdot 10^{-6}$. Because the feedback is a single sensor–actuator pair, $\mathbf{F}$ is rank one, and the matrix determinant lemma collapses (A.2) to the
 945 classical scalar return difference $1 + L(\lambda)$ of (A.1),

$$\mathcal{D}(\lambda) = 1 + c(\lambda) \mathbf{e}_N^\top \mathbf{Q}_0(\lambda)^{-1} \mathbf{e}_1, \quad (\text{A.3})$$

which replaces a full $N \times N$ solve *and* an $N \times N$ determinant by one solve with a single right-hand side: it agrees with the determinant form to $1.8 \cdot 10^{-16}$ relative and is 1.8 times cheaper per evaluation. For multi-input feedback of rank r the same lemma leaves an $r \times r$ determinant. One caveat is inherited from classical loop analysis: an exact pole–zero cancellation between the open loop and the feedback would remove that root from (A.2) – as it does from every input–
 950 output method – so the split should only be applied when the open-loop model is stabilizable and detectable through the chosen loop. The finite element chart reproduces the continuum result of A.8 within the resolved frequency range: the two are consistency checks on each other. As Section 3.1 argues, where such an open-loop/feedback split exists it is worth more than any tuning of the integrator.

A.10 Large dense determinant.. A synthetic stress test: $\mathcal{D}(\lambda) = \det(\lambda^2 \mathbf{M} + \lambda \mathbf{C} + \mathbf{K} + g e^{-\lambda\tau} \mathbf{K})$ with dense random
 955 symmetric positive definite 50×50 matrices, $\mathbf{C} = 0.25 \mathbf{K}$, swept over a delayed stiffness gain $g > -1$ and the delay τ . (At $g = -1$ exactly, the delayed stiffness cancels the static one and a characteristic root sits *on* the integration line; the package then reports the invalid-count marker instead of an arbitrary integer.) The evaluation cost is dominated by the LU factorization per frequency; the adaptive sweep keeps the number of evaluations low, so even this case yields charts in tens of seconds. This example also exercises the overflow guards of Sections 3.1–3.2: $|\mathcal{D}|$ reaches 10^{250}
 960 and beyond, so the phase integrand is evaluated in scale-invariant form and the leading-order probe (7) automatically retreats to a representable s_* .

A.11 Fractional-order oscillator.. $\mathcal{D}(\lambda) = \lambda^{1.8} + 0.5 \lambda^{0.8} + k e^{-\lambda\tau}$ over (k, τ) , with the principal branch of the fractional powers. The half-line count applies with the non-integer effective order estimated by (7); the origin offset of Section 3.1 regularizes the branch point at $\lambda = 0$. For $\sigma = 0$ the integration line stays right of the branch cut and the
 965 count is unambiguous; shifted lines with $\sigma < 0$ cross the cut of the principal branch and are *not* admissible – see the discussion at the end of this appendix (after A.12) for how the violation announces itself.

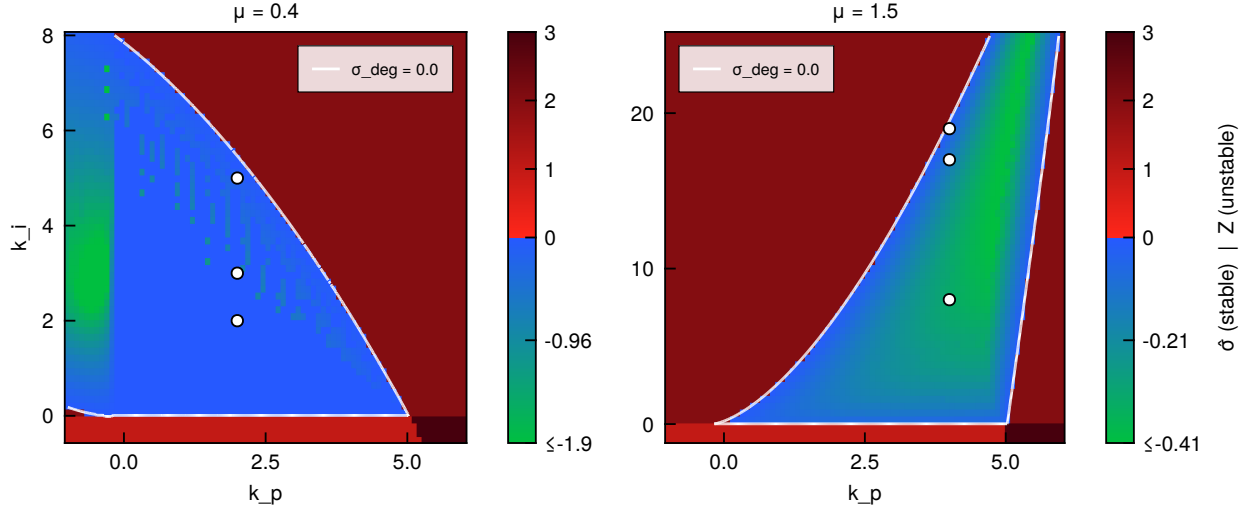


Figure A.12: Reproduction of a fractional-order controller design chart from the literature (Gao, Zhai and Liu [77], Example 1): stabilizing regions of the fractional PI controller $C(s) = k_p + k_i/s^\mu$ for the fractional delayed plant $G(s) = 5e^{-0.4s}/(10s^{0.5} + 1)$, for $\mu = 0.4$ (left) and $\mu = 1.5$ (right). White curve: the traced stability boundary ($\sigma = 0$); white markers: the test points tabulated in [77]. Only $\sigma = 0$ is shown, and deliberately so: the shifted line of Section 2.2 is *not* admissible for this class – see the text.

A.12 Fractional controller chart from the literature.. As an external validation of the fractional capability, Fig. A.12 reproduces the stabilizing-region chart of Gao, Zhai and Liu [77] (Example 1), computed there by D-decomposition; see also the related design charts [78, 79, 80, 81]. The closed-loop characteristic function $\mathcal{D}(s) = s^\mu(10s^{0.5} + 1) + 5e^{-0.4s}(k_p s^\mu + k_i)$ is a one-line model definition for the present method, and the traced boundary and the classification of the published test points agree with the reference.

Why only $\sigma = 0$, and how the violation announces itself.. Section 2.1 sets out why the imaginary axis is admissible for this class while a shifted line is not: a shifted line encloses part of the branch cut of s^μ and the argument principle no longer applies. The violation is detectable without theory: moving the line to the left can only ever *add* roots to its right, so the γ -stable regions must be *nested* – yet a 15×15 scan of both charts returns $Z(-0.5) < Z(0)$ at 3 of 450 points, and contours drawn at several σ levels cross, which no nested family can do. This is why the stability chart reproduces the literature while a stability-*degree* chart cannot be offered at all; extending the criterion to fractional systems with a prescribed decay rate requires a contour that respects the cut (a keyhole, or a formulation in the $w = s^{1/m}$ plane) and is left for future work.

Appendix B. The discrete phase-unwrapping march

This appendix lists the march of Section 3.6 in the form in which it is implemented (the function `calculate_unstable_roots_unwrap` of the package, and the GPU kernel of Section 8). All quantities are evaluated on the line $\lambda = \sigma + i\omega$; one call of $\mathcal{D}$ with a dual number $\omega + \varepsilon$ returns both $\mathcal{D}$ and $\mathcal{D}_\omega = i\mathcal{D}'$.

Sample.. For a sample at ω store the scale-free value $u = \mathcal{D}/s$, $s = \max\{|\operatorname{Re} \mathcal{D}|, |\operatorname{Im} \mathcal{D}|\}$ (which avoids overflow and underflow, essential in Float16), the phase slope $\theta' = \operatorname{Im}(\bar{u} \mathcal{D}_\omega)/(s|u|^2)$, and the sign $g = \operatorname{sgn} \operatorname{Re}(\bar{u} \mathcal{D}_\omega)$ of $\partial_\omega |\mathcal{D}|$.

Algorithm..

1. Set $\omega = \omega_0$ (10^{-9}), $\Phi = 0$, $h = h_0$ (10^{-2}); take the sample a at ω_0 . If $g_a > 0$, $|\mathcal{D}|$ has its minimum at $\omega = 0$: record a Newton root estimate from a .
2. Trial step $h_t = \min\{h, h_{\text{rel}} \max(\omega, 1), h_{\text{max}} \text{ if } \omega < \omega_{\text{band}}\}$, truncated at ω_{max} ; take the sample b at $\omega + h_t$.
3. Observed increment $\Delta = \operatorname{Arg}(u_b \bar{u}_a)$; error $e = |\Delta - \frac{h_t}{2}(\theta'_a + \theta'_b)|$; proposal $h_{\text{new}} = h_t \cdot \min\{4, \max\{0.2, 0.9(\text{tol}/e)^{1/3}\}\}$.
4. If $e \leq \text{tol}$: accept. If $e > \text{tol}$ but $h_{\text{new}} \leq 8\epsilon_{\text{mach}} \max(\omega, 1)$ (sub-resolution transition): compute the Newton step $q = \mathcal{D}/\mathcal{D}_\omega$ at b ; if $|\Delta| > \pi/2$, take the branch $\Delta - 2\pi$ when $\Delta > 0$ and the root lies right of the line ($\operatorname{Im} q > 0$), $\Delta + 2\pi$ when $\Delta < 0$ and the root lies left of it; accept and flag the point. Otherwise reject, set $h = h_{\text{new}}$ and go to step 2.
5. On acceptance: if $g_a < 0 < g_b$, a minimum of $|\mathcal{D}|$ lies inside the step; locate it on the cubic Hermite interpolant of $\mathcal{D}$ through $(\mathcal{D}_a, \mathcal{D}_{\omega,a}, \mathcal{D}_b, \mathcal{D}_{\omega,b})$ by four regula-falsi (Illinois) iterations on $\operatorname{Re}(\bar{p} p') = 0$, take one Newton step from it, and insert the estimate $(\hat{\sigma}, \hat{\omega})$ into the depth-sorted buffer of the N deepest minima if it lies inside the trust region. Then $\Phi \leftarrow \Phi + \Delta$, $a \leftarrow b$, $h = h_{\text{new}}$.
6. Repeat from step 2 until $\omega = \omega_{\text{max}}$; return $\tilde{Z} = n/2 - \Phi/\pi$, $Z = \text{round}(\tilde{Z})$, the root estimates, and the number of evaluations.

Parameters and flags.. The defaults are $\text{tol} = 0.3$ rad, $\omega_{\text{max}} = 10^5$, $h_0 = 10^{-2}$, $h_{\text{rel}} = 1$, and no band cap ($h_{\text{max}} = \infty$). The tolerance is a statement about the phase only: it is independent of the scale of $\mathcal{D}$, and it is far from critical: every value from 0.05 to 1.2 rad reproduces the three charts of Table 9 without a single wrong count, while the cost changes by at most a factor of 2.5 between 0.1 and 0.5 – a skipped transition costs an error of almost π , well above any sensible tolerance. The GPU kernel returns three flag bits per point: *failed* (the step fell below the resolution without a decidable branch, or the step budget was exhausted), *decided* (step 4 fired) and *residual* ($|\tilde{Z} - Z| > 1/4$, typically a root on the line). Flagged points are the candidates for a re-check in a wider number format.

References

- [1] S. A. Tobias, W. Fishwick, Theory of regenerative machine tool chatter, *The Engineer* 205 (1958) 199–203, 238–239, london.
- [2] J. Tlustý, M. Polacek, The stability of machine tools against self-excited vibrations in machining, in: *International Research in Production Engineering*, ASME, 1963, pp. 465–474.
- [3] Y. Altintas, E. Budak, Analytical prediction of stability lobes in milling, *CIRP Annals - Manufacturing Technology* 44 (1) (1995) 357–362. doi:10.1016/S0007-8506(07)62342-7.
- [4] J. Munoa, X. Beudaert, Z. Dombovari, Y. Altintas, E. Budak, C. Brecher, G. Stépán, Chatter suppression techniques in metal cutting, *CIRP Annals - Manufacturing Technology* 65 (2) (2016) 785–808. doi:10.1016/j.cirp.2016.06.004.
- [5] R. Bellman, K. L. Cooke, *Differential-Difference Equations*, Vol. 6 of *Mathematics in Science and Engineering*, Academic Press, New York, 1963.

- [6] G. Stépán, Retarded Dynamical Systems: Stability and Characteristic Functions, Vol. 210 of Pitman Research Notes in Mathematics Series, Longman Scientific & Technical, Harlow, 1989, co-published with John Wiley & Sons, New York.
- [7] J. K. Hale, S. M. Verduyn Lunel, Introduction to Functional Differential Equations, Vol. 99 of Applied Mathematical Sciences, Springer, New York, 1993. doi:10.1007/978-1-4612-4342-7.
- [8] W. Michiels, S.-I. Niculescu, Stability, Control, and Computation for Time-Delay Systems: An Eigenvalue-Based Approach, 2nd Edition, Advances in Design and Control, SIAM, Philadelphia, PA, 2014. doi:10.1137/1.9781611973631.
- [9] T. Insperger, G. Stépán, Semi-Discretization for Time-Delay Systems: Stability and Engineering Applications, Vol. 178 of Applied Mathematical Sciences, Springer, New York, 2011. doi:10.1007/978-1-4614-0335-7.
- [10] R. Sipahi, S.-I. Niculescu, C. T. Abdallah, W. Michiels, K. Gu, Stability and stabilization of systems with time delay: Limitations and opportunities, IEEE Control Systems Magazine 31 (1) (2011) 38–65. doi:10.1109/MCS.2010.939135.
- [11] T. Insperger, G. Stépán, Semi-discretization method for delayed systems, International Journal for Numerical Methods in Engineering 55 (5) (2002) 503–518. doi:10.1002/nme.505.
- [12] T. Insperger, G. Stépán, Updated semi-discretization method for periodic delay-differential equations with discrete delay, International Journal for Numerical Methods in Engineering 61 (1) (2004) 117–141. doi:10.1002/nme.1061.
- [13] T. Insperger, G. Stépán, J. Turi, On the higher-order semi-discretizations for periodic delayed systems, Journal of Sound and Vibration 313 (1–2) (2008) 334–341. doi:10.1016/j.jsv.2007.11.040.
- [14] Y. Ding, L. Zhu, X. Zhang, H. Ding, A full-discretization method for prediction of milling stability, International Journal of Machine Tools and Manufacture 50 (5) (2010) 502–509. doi:10.1016/j.ijmachtools.2010.01.003.
- [15] E. A. Butcher, H. Ma, E. Bueler, V. Averina, Z. Szabó, Stability of linear time-periodic delay-differential equations via Chebyshev polynomials, International Journal for Numerical Methods in Engineering 59 (7) (2004) 895–922. doi:10.1002/nme.894.
- [16] D. Breda, S. Maset, R. Vermiglio, Pseudospectral differencing methods for characteristic roots of delay differential equations, SIAM Journal on Scientific Computing 27 (2) (2005) 482–495. doi:10.1137/030601600.
- [17] D. Breda, S. Maset, R. Vermiglio, Stability of Linear Delay Differential Equations: A Numerical Approach with MATLAB, SpringerBriefs in Control, Automation and Robotics, Springer, New York, NY, 2015. doi:10.1007/978-1-4939-2107-2.
- [18] F. A. Khasawneh, B. P. Mann, A spectral element approach for the stability of delay systems, International Journal for Numerical Methods in Engineering 87 (6) (2011) 566–592. doi:10.1002/nme.3122.
- [19] Z. Wu, W. Michiels, Reliably computing all characteristic roots of delay differential equations in a given right half plane using a spectral method, Journal of Computational and Applied Mathematics 236 (9) (2012) 2499–2514. doi:10.1016/j.cam.2011.12.009.
- [20] K. Engelborghs, T. Luzyanina, D. Roose, Numerical bifurcation analysis of delay differential equations using DDE-BIFTOOL, ACM Transactions on Mathematical Software 28 (1) (2002) 1–21. doi:10.1145/513001.513002.
- [21] J. Sieber, K. Engelborghs, T. Luzyanina, G. Samaey, D. Roose, DDE-BIFTOOL manual — bifurcation analysis of delay differential equations, arXiv:1406.7144v4 (2016). arXiv:1406.7144.
URL <https://arxiv.org/abs/1406.7144>
- [22] D. Breda, S. Maset, R. Vermiglio, TRACE-DDE: a tool for robust analysis and characteristic equations for delay differential equations, in: J. J. Loiseau, W. Michiels, S.-I. Niculescu, R. Sipahi (Eds.), Topics in Time Delay Systems: Analysis, Algorithms and Control, Vol. 388 of Lecture Notes in Control and Information Sciences, Springer, Berlin, Heidelberg, 2009, pp. 145–155. doi:10.1007/978-3-642-02897-7_13.
- [23] R. Szalai, G. Stépán, S. J. Hogan, Continuation of bifurcations in periodic delay-differential equations using characteristic matrices, SIAM Journal on Scientific Computing 28 (4) (2006) 1301–1317. doi:10.1137/040618709.
- [24] R. Szalai, Knut: a continuation and bifurcation software for delay-differential equations, source code at <https://github.com/rs1909/knut>, accessed 2026-07-16 (2013).
URL <https://rs1909.github.io/knut/>
- [25] D. Bachrathy, H. T. Sykora, SemiDiscretizationMethod.jl: A Julia package to investigate the behaviour of linear delay differential equations, version 0.4.1, accessed 2026-07-16 (2026).

URL <https://github.com/bachrathyd/SemiDiscretizationMethod.jl>

- [26] D. Bachrathy, Linear time complexity analysis of time-periodic delayed systems with multiplication free semi-discretization method, *Journal of Vibration and Control* Published online 22 May 2026 (2026). doi:10.1177/10775463261453846.
- [27] J. K. Hale, S. M. Verduyn Lunel, Strong stabilization of neutral functional differential equations, *IMA Journal of Mathematical Control and Information* 19 (1–2) (2002) 5–23. doi:10.1093/imamci/19.1_and_2.5.
- [28] W. Michiels, T. Vyhldal, An eigenvalue based approach for the stabilization of linear time-delay systems of neutral type, *Automatica* 41 (6) (2005) 991–998. doi:10.1016/j.automatica.2004.11.032.
- [29] S. L. Campbell, *Singular Systems of Differential Equations*, Vol. 40 of Research Notes in Mathematics, Pitman, San Francisco, 1980.
- [30] P. Ha, V. Mehrmann, Analysis and numerical solution of linear delay differential-algebraic equations, *BIT Numerical Mathematics* 56 (2) (2016) 633–657. doi:10.1007/s10543-015-0577-6.
- [31] J. M. Cushing, *Integrodifferential Equations and Delay Models in Population Dynamics*, Vol. 20 of Lecture Notes in Biomathematics, Springer, Berlin, Heidelberg, 1977. doi:10.1007/978-3-642-93073-7.
- [32] I. Petráš, *Fractional-Order Nonlinear Systems: Modeling, Analysis and Simulation*, Nonlinear Physical Science, Springer, Berlin, Heidelberg, 2011. doi:10.1007/978-3-642-18101-6.
- [33] A. R. Fioravanti, C. Bonnet, H. Özbay, S.-I. Niculescu, A numerical method for stability windows and unstable root-locus calculation for linear fractional time-delay systems, *Automatica* 48 (11) (2012) 2824–2830. doi:10.1016/j.automatica.2012.04.009.
- [34] J. Čermák, T. Kisela, Delay-dependent stability switches in fractional differential equations, *Communications in Nonlinear Science and Numerical Simulation* 79 (2019) 104888. doi:10.1016/j.cnsns.2019.104888.
- [35] K. Gu, V. L. Kharitonov, J. Chen, *Stability of Time-Delay Systems*, Birkhäuser, Boston, 2003. doi:10.1007/978-1-4612-0039-0.
- [36] J. I. Neimark, D-decomposition of the space of quasi-polynomials (on the stability of linearized distributive systems), *Prikladnaya Matematika i Mekhanika* 13 (1949) 349–380, in Russian. English translation: American Mathematical Society Translations, Series 2, vol. 102, pp. 95–131, 1973.
- [37] H. Nyquist, Regeneration theory, *Bell System Technical Journal* 11 (1) (1932) 126–147. doi:10.1002/j.1538-7305.1932.tb02344.x.
- [38] A. V. Mikhailov, Metod garmonicheskogo analiza v teorii regulirovaniya (Method of harmonic analysis in control theory), *Avtomatika i Telemekhanika* (3) (1938) 27–81, in Russian.
- [39] K. Walton, J. E. Marshall, Direct method for TDS stability analysis, *IEE Proceedings D – Control Theory and Applications* 134 (2) (1987) 101–107. doi:10.1049/ip-d.1987.0018.
- [40] Z. V. Rekasius, A stability test for systems with delays, in: *Proceedings of the Joint Automatic Control Conference*, San Francisco, CA, 1980, paper No. TP9-A.
- [41] N. Olgac, R. Sipahi, An exact method for the stability analysis of time-delayed linear time-invariant (LTI) systems, *IEEE Transactions on Automatic Control* 47 (5) (2002) 793–797. doi:10.1109/TAC.2002.1000275.
- [42] X.-G. Li, S.-I. Niculescu, A. Çela, *Analytic Curve Frequency-Sweeping Stability Tests for Systems with Commensurate Delays*, SpringerBriefs in Electrical and Computer Engineering – Control, Automation and Robotics, Springer, Cham, 2015. doi:10.1007/978-3-319-15717-7.
- [43] S. Yi, P. W. Nelson, A. G. Ulsoy, *Time-Delay Systems: Analysis and Control Using the Lambert W Function*, World Scientific, Singapore, 2010. doi:10.1142/7759.
- [44] J. Louisell, A matrix method for determining the imaginary axis eigenvalues of a delay system, *IEEE Transactions on Automatic Control* 46 (12) (2001) 2008–2012. doi:10.1109/9.975510.
- [45] T. Vyhldal, P. Zitek, Mapping based algorithm for large-scale computation of quasi-polynomial zeros, *IEEE Transactions on Automatic Control* 54 (1) (2009) 171–177. doi:10.1109/TAC.2008.2008345.
- [46] T. Vyhldal, P. Zitek, QPmR – Quasi-Polynomial Root-Finder: Algorithm update and examples, in: T. Vyhldal, J.-F. Lafay, R. Sipahi (Eds.), *Delay Systems: From Theory to Numerics and Applications*, Vol. 1 of Advances in Delays and Dynamics, Springer, Cham, 2014, pp. 299–312. doi:10.1007/978-3-319-01695-5_22.

- [47] W.-J. Beyn, An integral method for solving nonlinear eigenvalue problems, *Linear Algebra and its Applications* 436 (10) (2012) 3839–3863. doi:10.1016/j.laa.2011.03.030.
- [48] D. Avanesoff, A. R. Fioravanti, C. Bonnet, YALTA: a Matlab toolbox for the H_∞ -stability analysis of classical and fractional systems with commensurate delays, in: 11th IFAC Workshop on Time-Delay Systems, Vol. 46 of IFAC Proceedings Volumes, 2013, pp. 839–844. doi:10.3182/20130204-3-FR-4032.00088.
- [49] S.-I. Niculescu, I. Boussaada, Counting characteristic roots of linear delay differential equations. Part II: From argument principle to rightmost root assignment methods, in: D. Breda (Ed.), *Controlling Delayed Dynamics: Advances in Theory, Methods and Applications*, Vol. 604 of CISM International Centre for Mechanical Sciences, Springer, Cham, 2023, pp. 157–193. doi:10.1007/978-3-031-01129-0_6.
- [50] D. Bachrathy, G. Stépán, Bisection method in higher dimensions and the efficiency number, *Periodica Polytechnica Mechanical Engineering* 56 (2) (2012) 81–86. doi:10.3311/pp.me.2012-2.01.
- [51] D. Bachrathy, MDBM.jl: Multi-dimensional bisection method, version 0.1.7 (2025). URL <https://github.com/bachrathyd/MDBM.jl>
- [52] G. Stépán, Modelling nonlinear regenerative effects in metal cutting, *Philosophical Transactions of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences* 359 (1781) (2001) 739–757. doi:10.1098/rsta.2000.0753.
- [53] T. Insperger, B. P. Mann, G. Stépán, P. V. Bayly, Stability of up-milling and down-milling, part I: Alternative analytical methods, *International Journal of Machine Tools and Manufacture* 43 (1) (2003) 25–34. doi:10.1016/S0890-6955(02)00159-1.
- [54] D. Bachrathy, G. Stépán, Improved prediction of stability lobes with extended multi frequency solution, *CIRP Annals - Manufacturing Technology* 62 (1) (2013) 411–414. doi:10.1016/j.cirp.2013.03.085.
- [55] V. B. Kolmanovskii, A. Myshkis, *Introduction to the Theory and Applications of Functional Differential Equations*, Vol. 463 of Mathematics and Its Applications, Kluwer Academic Publishers, Dordrecht, 1999. doi:10.1007/978-94-017-1965-0.
- [56] L. M. Delves, J. N. Lyness, A numerical method for locating the zeros of an analytic function, *Mathematics of Computation* 21 (100) (1967) 543–560. doi:10.1090/S0025-5718-1967-0228165-4.
- [57] P. Kravanja, M. Van Barel, *Computing the Zeros of Analytic Functions*, Vol. 1727 of Lecture Notes in Mathematics, Springer, Berlin, Heidelberg, 2000. doi:10.1007/BFb0103927.
- [58] P. Kravanja, M. Van Barel, O. Ragos, M. N. Vrahatis, F. A. Zafiropoulos, ZEAL: A mathematical software package for computing zeros of analytic functions, *Computer Physics Communications* 124 (2–3) (2000) 212–232. doi:10.1016/S0010-4655(99)00429-4.
- [59] B. D. Hassard, Counting roots of the characteristic equation for linear delay-differential systems, *Journal of Differential Equations* 136 (2) (1997) 222–235. doi:10.1006/jdeq.1996.3127.
- [60] P. Zítek, T. Vyhlídal, Argument-increment based stability criterion for neutral time delay systems, in: *Proceedings of the 16th Mediterranean Conference on Control and Automation*, 2008, pp. 824–829. doi:10.1109/MED.2008.4602199.
- [61] T. Vyhlídal, P. Zítek, Modification of Mikhaylov criterion for neutral time-delay systems, *IEEE Transactions on Automatic Control* 54 (10) (2009) 2430–2435. doi:10.1109/TAC.2009.2029301.
- [62] Q. Xu, Z. Wang, Exact stability test of neutral delay differential equations via a rough estimation of the testing integral, *International Journal of Dynamics and Control* 2 (2) (2014) 154–163. doi:10.1007/s40435-013-0044-7.
- [63] Q. Xu, G. Stépán, Z. Wang, Delay-dependent stability analysis by using delay-independent integral evaluation, *Automatica* 70 (2016) 153–157. doi:10.1016/j.automatica.2016.03.028.
- [64] J. H. Verner, Numerically optimal Runge–Kutta pairs with interpolants, *Numerical Algorithms* 53 (2–3) (2010) 383–396. doi:10.1007/s11075-009-9290-3.
- [65] C. Tsitouras, Runge–Kutta pairs of order 5(4) satisfying only the first column simplifying assumption, *Computers & Mathematics with Applications* 62 (2) (2011) 770–775. doi:10.1016/j.camwa.2011.06.002.
- [66] C. Rackauckas, Q. Nie, DifferentialEquations.jl – a performant and feature-rich ecosystem for solving differential equations in Julia, *Journal of Open Research Software* 5 (1) (2017) 15. doi:10.5334/jors.151.
- [67] J. Revels, M. Lubin, T. Papamarkou, Forward-mode automatic differentiation in Julia (2016). arXiv:1607.07892.

URL <https://arxiv.org/abs/1607.07892>

[68] J. Bezanson, A. Edelman, S. Karpinski, V. B. Shah, Julia: A fresh approach to numerical computing, *SIAM Review* 59 (1) (2017) 65–98. doi:10.1137/141000671.

1150 [69] L. Zhang, G. Stépán, Exact stability chart of an elastic beam subjected to delayed feedback, *Journal of Sound and Vibration* 367 (2016) 219–232. doi:10.1016/j.jsv.2016.01.002.

[70] W. Michiels, Spectrum-based stability analysis and stabilisation of systems described by delay differential algebraic equations, *IET Control Theory & Applications* 5 (16) (2011) 1829–1842. doi:10.1049/iet-cta.2010.0752.

[71] S. G. Johnson, QuadGK.jl: Gauss–Kronrod integration in Julia, version 2.11.3 (2013).

1155 URL <https://github.com/JuliaMath/QuadGK.jl>

[72] H. H. Rosenbrock, Some general implicit processes for the numerical solution of differential equations, *The Computer Journal* 5 (4) (1963) 329–330. doi:10.1093/comjnl/5.4.329.

[73] L. F. Shampine, M. W. Reichelt, The MATLAB ODE suite, *SIAM Journal on Scientific Computing* 18 (1) (1997) 1–22. doi:10.1137/S1064827594276424.

1160 [74] E. Hairer, G. Wanner, Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems, 2nd Edition, Vol. 14 of Springer Series in Computational Mathematics, Springer, Berlin, Heidelberg, 1996. doi:10.1007/978-3-642-05221-7.

[75] D. J. VandenHeuvel, DelaunayTriangulation.jl: A Julia package for Delaunay triangulations and Voronoi tessellations in the plane, *Journal of Open Source Software* 9 (101) (2024) 7174. doi:10.21105/joss.07174.

[76] M. Kidd, G. Stépán, Delayed control of an elastic beam, *International Journal of Dynamics and Control* 2 (1) (2014) 68–76. doi:10.1007/s40435-014-0079-4.

1165 [77] Z. Gao, L.-R. Zhai, Y.-D. Liu, Robust stabilizing regions of fractional-order PI^l controllers for fractional-order systems with time-delays, *International Journal of Automation and Computing* 14 (3) (2017) 340–349. doi:10.1007/s11633-015-0941-7.

[78] S. E. Hamamci, An algorithm for stabilization of fractional-order time delay systems using fractional-order PID controllers, *IEEE Transactions on Automatic Control* 52 (10) (2007) 1964–1969. doi:10.1109/TAC.2007.906243.

1170 [79] N. Hohenbichler, Comments on “An algorithm for stabilization of fractional-order time delay systems using fractional-order PID controllers”, *IEEE Transactions on Automatic Control* 53 (11) (2008) 2712. doi:10.1109/TAC.2008.2007535.

[80] A. Ruszewski, Stability regions of closed loop system with time delay inertial plant of fractional order and fractional order PI controller, *Bulletin of the Polish Academy of Sciences: Technical Sciences* 56 (4) (2008) 329–332.

[81] Y.-C. Cheng, C. Hwang, Stabilization of unstable first-order time-delay systems using fractional-order PD controllers, *Journal of the Chinese Institute of Engineers* 29 (2) (2006) 241–249. doi:10.1080/02533839.2006.9671121.

1175

Table 11: Comparison with semi-discretization (`SemiDiscretizationMethod.jl`, orders 1 and 2, n steps per delay) on the reduced showcase system: error of the dominant characteristic exponent and CPU time per parameter point. The reference is the Newton-polished dominant root of the quasi-polynomial itself, certified independently of either sweep by its recorded defect $|\mathcal{D}(\lambda_{\text{ref}})| < 10^{-12}$. The unrefined rows report the accuracy of the tracked estimate alone; the refined rows share the polishing machinery with the reference, so their error reflects the polish rather than the sweep.

point	method / setting	error of $\text{Re } \lambda_{\text{dom}}$	time / point
stable	SD order 1, $n = 20$	$2.9 \cdot 10^{-5}$	5.1 ms
	SD order 1, $n = 160$	$4.6 \cdot 10^{-7}$	457.9 ms
	SD order 2, $n = 20$	$2.0 \cdot 10^{-6}$	6.5 ms
	SD order 2, $n = 160$	$4.0 \cdot 10^{-9}$	477.7 ms
	this work, tol 10^{-5} , unrefined	$2.7 \cdot 10^{-4}$	1.3 ms
	this work, tol 10^{-5}	$6.2 \cdot 10^{-17}$	1.4 ms
	this work, tol 10^{-8}	$3.1 \cdot 10^{-17}$	3.0 ms
near boundary	SD order 1, $n = 20$	$9.3 \cdot 10^{-5}$	5.3 ms
	SD order 1, $n = 160$	$1.5 \cdot 10^{-6}$	449.8 ms
	SD order 2, $n = 20$	$4.0 \cdot 10^{-6}$	5.5 ms
	SD order 2, $n = 160$	$7.9 \cdot 10^{-9}$	515.2 ms
	this work, tol 10^{-5} , unrefined	$2.7 \cdot 10^{-4}$	1.7 ms
	this work, tol 10^{-5}	$6.9 \cdot 10^{-18}$	1.8 ms
	this work, tol 10^{-8}	$6.9 \cdot 10^{-18}$	4.1 ms
unstable	SD order 1, $n = 20$	$9.7 \cdot 10^{-5}$	9.7 ms
	SD order 1, $n = 160$	$1.5 \cdot 10^{-6}$	434.1 ms
	SD order 2, $n = 20$	$2.5 \cdot 10^{-7}$	9.3 ms
	SD order 2, $n = 160$	$5.6 \cdot 10^{-10}$	447.3 ms
	this work, tol 10^{-5} , unrefined	$3.6 \cdot 10^{-2}$	1.2 ms
	this work, tol 10^{-5}	$5.5 \cdot 10^{-11}$	1.2 ms
	this work, tol 10^{-8}	$4.1 \cdot 10^{-12}$	2.8 ms

Table 12: Evaluations of $\mathcal{D}$ needed for one parameter point at $\text{tol} = 10^{-5}$, with the count of each entry verified against a tight-tolerance reference before being recorded. What governs the cost is how fast the delay-induced ripple of θ' decays, i.e. how far the delayed terms sit below the leading power: ω^{-1} for the velocity-feedback showcase, ω^{-3} for the fourth-order position-feedback benchmark.

system	back-end	$\omega_{\max} = 10^4$	$\omega_{\max} = 10^6$
showcase (ω^{-1} ripple)	ordered march	11 138	578 418
	quadrature	15 315	1 675 500
4th-order (ω^{-3} ripple)	ordered march	1 330	1 426
	quadrature	555	570

Table 13: The quantities behind Fig. 9: the cost of one point on one thread, the per-point cost of the saturated device, and their ratio, the *effective parallelism* P_{eff} – the number of single-thread marches the device completes in the time of one. For the GPUs the number of CUDA cores is given for comparison, with $P_{\text{eff}}/\text{cores}$ in parentheses.

hardware	format	one thread [$\mu\text{s}/\text{point}$]	saturated [ns/point]	effective parallelism	CUDA cores (ratio)
CPU desktop, i5-10400 (6 cores)	Float64	4.24	485	8.7	12 threads
CPU server (G4 host, 48 threads)	Float64	3.79	100	38.0	48 threads
GPU T4	Float32	30.0	6.29	4 763	2560 (1.86)
GPU T4	Float64	214.8	143.29	1 499	2560 (0.59)
GPU L4	Float32	21.2	2.37	8 929	7424 (1.20)
GPU L4	Float64	172.7	77.76	2 221	7424 (0.30)
GPU A100	Float32	31.8	1.80	17 637	6912 (2.55)
GPU A100	Float64	60.9	5.37	11 345	6912 (1.64)
GPU RTX PRO 6000	Float32	20.7	0.48	42 953	24064 (1.78)
GPU RTX PRO 6000	Float64	271.6	20.03	13 563	24064 (0.56)

Table 14: Full-HD brute-force charts ($1920 \times 1080 \approx 2.07 \cdot 10^6$ points, one GPU thread per point) of the fourth-order benchmark with the discrete march on four NVIDIA GPUs (Google Colab): median kernel time in milliseconds over ten repetitions. $\omega_{\max} = 10^5$ is the production window; $\omega_{\max} = 15$ suffices for this model and is required by Float16, whose range ends at 65504; in the Float16 column $\mathcal{D}$ is evaluated in Float16 on a rescaled frequency axis while the march runs in Float32. Float64 and Float32 give identical charts; the last column is the Float16 throughput in million points per second.

GPU	$\omega_{\max} = 10^5$		$\omega_{\max} = 15$			Float16 [Mpts/s]
	Float64	Float32	Float64	Float32	Float16	
T4	195.95	50.06	126.88	35.48	6.12	339
L4	102.64	9.04	66.41	5.31	2.78	746
A100 (80 GB)	10.17	4.93	5.55	2.78	2.29	904
RTX PRO 6000 (Blackwell)	27.65	2.84	17.96	1.68	0.74	2816

Table 15: The discrete march with $\mathcal{D}$ evaluated in narrow number formats (400×400 chart of the fourth-order benchmark, $\omega_{\max} = 15$, best of four frequency scalings $\lambda = \Omega\mu$ per format; the march itself runs in Float32). *Flagged* points carry a sub-resolution or residual flag and are re-checked in a wider format; *failed* points did not reach $\omega_{\max}$. *Emulated: every operation is rounded to the format.

format of $\mathcal{D}$	bits (exp./mant.)	wrong Z [%]	flagged [%]	failed [%]
Float64	11/52	0.000	0.00	0.00
Float32	8/23	0.000	0.00	0.00
Float16	5/10	0.014	1.74	0.00
BFloat16*	8/7	0.171	14.05	0.00
Float8 E5M2*	5/2	8.780	98.38	3.00
Float8 E4M3*	4/3	9.086	95.89	1.59
Float4 E2M1*	2/1	100.000	100.00	10.56